

Министерство образования Свердловской области
ГАПОУ СО «Екатеринбургский колледж транспортного строительства»

Специальность 09.02.07
«Информационные системы и программирование»

Заведующий отделением

_____ Е.В.Дудель

Председатель цикловой комиссии

_____ Г.В.Мирошниченко

Разработка автоматизированной системы для кафе

Пояснительная записка

к дипломному проекту

ДП-ПР-41-21-2026-ПЗ

Разработал:

Студент гр. ПР-41

_____ / М. К. Шмелев

Руководитель:

_____ / С. И. Овчинникова

Н. контроль:

_____ / С. И. Овчинникова

Министерство образования Свердловской области
ГАПОУ СО «Екатеринбургский колледж транспортного строительства»

Специальность 09.02.07
«Информационные системы и программирование»

Разработка автоматизированной системы для кафе

Дипломный проект

ДП-ПР-41-21-2026-ПЗ

Министерство образования Свердловской области
ГАПОУ СО «Екатеринбургский колледж транспортного строительства»

Специальность **09.02.07 Информационные системы и программирование**

Рассмотрено:
на заседании ЦМК
протокол № 7
от «20» марта 2026г.

председатель ЦМК

_____/Г.В. Мирошниченко

Утверждаю:
зам. директора по
УВР ГАПОУ СО «ЕКТС»

_____/А.М. Шанин

ЗАДАНИЕ
НА ДИПЛОМНЫЙ ПРОЕКТ

Студенту Шмелеву Максиму Константиновичу

Группа ПР-41

Тема: Разработка автоматизированной системы для кафе

Руководитель: Овчинникова Светлана Ивановна

Срок дипломного проектирования:

С 18.05.2026г. по 28.06.2026г.

Сдача проекта на рецензию 13.06.2026г.2026

СОСТАВ ДИПЛОМНОГО ПРОЕКТА

Задание на дипломное проектирование

Отзыв руководителя

Рецензия

Пояснительная записка

Программный продукт (при отсутствии грифа «секретно»)

РЕКОМЕНДУЕМОЕ СОДЕРЖАНИЕ ПОЯСНИТЕЛЬНОЙ ЗАПИСКИ

- Титульный лист
- Задание дипломного проектирования
- Содержание
- Введение - *оценка современного состояния проблемы, решаемой в проекте, обоснование необходимости выполнения проекта, новизна и актуальность темы*
- Основная часть;
 - 1. Описание предметной области
 - 2. Назначение и область применения программы- *описываются и анализируются объект и предмет исследования, обосновывается выбор применяемых средств, методов, технологий;*
 - 3. Проектирование задачи- *предлагаются решения поставленных задач (проектирование и разработка базы данных, проектирование и разработка пользовательского интерфейса, разработка программ, разработка эксплуатационных документов – инструкций, руководств);*
 - 3.1 Обоснование инструментов разработки
 - 3.2 Описание алгоритма решения задачи
 - 4. Программа решения задачи
 - 4.1 Логическая структура
 - 4.2 Физическая структура
 - 5. Тестирование и отладка программы
 - 6. Применение
 - 6.1 Назначение программы
 - 6.2 Требования к аппаратным ресурсам ПК
 - 6.3 Руководство пользователя
 - 7. Охрана труда и противопожарная безопасность
Заключение - краткие выводы о результатах выполненной работы, оценка перспектив дальнейшего улучшения функциональности проекта, предложения по использованию результатов работы.
- Список используемых источников;

1. Макдональд М. WPF: Windows Presentation Foundation в .NET 4.5 с примерами на C# 5.0 для профессионалов [Текст] / М. Макдональд. – Москва: Вильямс, 2013. – 1024 с.

2. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения [Текст] / Р. Мартин. – Санкт-Петербург: Питер, 2021. – 352 с.
3. Прайс М. С# 9 и .NET 5. Разработка и оптимизация [Текст] / М. Прайс. - Санкт-Петербург: Питер, 2022. – 848 с.
4. Троелсен Э. Язык программирования С# 9 и платформа .NET 5 [Текст] / Э. Троелсен, Ф. Джепикс. – Москва: Вильямс, 2022. – 1328 с.
5. Фримен А. ASP.NET Core для профессионалов [Текст] / А. Фримен. – Москва: Вильямс, 2019. – 912 с.
6. Документация по службе «Управление API» [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/azure/api-management/> (дата обращения 10.10.2025).
7. Документация по ASP.NET Core [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/aspnet/core/> (дата обращения 10.11.2025).
8. Документация по Entity Framework Core [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/ef/core/> (дата обращения 11.11.2025).
9. Руководство по языку С# [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/csharp/> (дата обращения 13.01.2026).

- Приложения

Приложение А Схема алгоритма

Приложение Б Текст программы

Приложение В Структура данных

Приложение Г Скриншоты клиентского приложения

График выполнения ДИПЛОМНОГО ПРОЕКТА

№ п/п	Наименование работ	Содержание	Выполнен ие работ в %	Список исполнения
1.	Выбор, выдача темы	Тема работы		08.04.2026
2.	Внешнее проектирование (схемы проверки программы, тесты, контроль целостности данных)	Спецификация программы и данных	20	20.05.2026
3.	Проектирование (укрупнённый проект) а) Функциональная схема б) Схемы данных в) Структурная схема г) Схема пользовательского интерфейса д) Укрупнённый алгоритм программы Детальное проектирование Оформлены разделы в пояснительной записке (введение, Описание предметной области) Разработка в среде программирования а) Ядро программы б) Функциональная часть в) Пользовательский интерфейс	Готовая функциональная часть	60	03.06.2026
4.	а) Защитная часть (авторизация, журнал пользователей, шифрование и т.д.) б) Сервисная часть (настройки, справки и т.д.) Оформлены разделы в пояснительной записке (Проектирование задачи, Программа решения задачи) Программа написана в стадии отладки. Тестирование и анализ результатов Оформлены разделы в пояснительной записке (Тестирование, Применение) Оформлена пояснительная записка полностью	Готовая пояснительная записка	90	10.06.2026
5.	Оформление экспериментальной части, подготовка демонстрационных данных и проекта в целом Подготовка к защите проекта (отзыв, рецензия, доклад)	Получены отзывы и рецензии	100	14.06.2026
6.	Нормоконтроль			14.06.2026
7.	Защита дипломных проектов			22.06.2026 28.06.2026

Контроль проверки:

20%	20.05.2026
60%	03.06.2026
90%	10.06.2026
100%	14.06.2026

ИСХОДНЫЕ ДАННЫЕ

(заполняется руководителем проекта)

В ходе выполнения дипломного проекта необходимо реализовать:

1. Проектировка базы данных, удовлетворяющих потребностям:
 - a. Управление каталогом: хранение и данных о блюдах, состава блюд, категории, заказы.
 - b. Система пользователей: реализация ролевой модели (Администратор, Официант, Повар)
 - c. Индексацию и поиск: быстрая выборка заказов по статусам посредством кеширования.
 - d. Складской учет: компоненты состава блюд.
2. Разработка десктопного приложения для персонала, включающее:
 - a. Каталог: получение актуального меню с ценами и описанием;
 - b. Детальная информация: просмотр каждого блюда, и его состав
 - c. Оформление заказа: создание нового заказа с набором блюд и привязкой к столику.
 - d. История: просмотр архива выполненных заказов
 - e. Просмотр активных заказов с таймером приготовления
3. Разработка функционала для администратора:
 - a. Управление контентом: добавление новых позиций, редактирование, удаление из меню.
 - b. Регистрация новых сотрудников, обновление данных, удаление;
 - c. Формирование отчета по выручке за день и месяц с возможностью экспорта документов.
4. Функционал для повара:
 - a. Мгновенное получение информации о новых заказах и передача их в работу
 - b. Мониторинг очереди: получение списка всех активных заказов, которые находятся в статусе «Ожидает» или «Готовится».
 - c. Детализация производства: просмотр каждого поступившего заказа (наименования блюд, количество порций и модификаторы) для корректной сборки.
 - d. Управление статусами готовности: смена статуса заказа для автоматического уведомления официантов о необходимости выдачи блюда.
5. Обеспечение надежности и развертывание системы
 - a. Автоматизированное тестирование: разработка комплекса интеграционных тестов для проверки бизнес-сценариев.

- б. Контейнеризация среды: упаковка серверного приложения и базы данных в изолированные Docker-контейнеры для обеспечения быстрого и кроссплатформенного развертывания системы.

Задание выдано

06.04.2026 г.

Дипломник

_____/Шмелев М.К.

Руководитель Дипломного проекта

_____/ Овчинникова С.И.

Данные руководителя (ВУЗ, год окончания ВУЗа, инженерный стаж после окончания ВУЗа)

Уральский политехнический институт, радиотехнический факультет, 1992 г., стаж работы по специальности 32 года.

Содержание

Введение.....	9
1 Описание предметной области	11
2 Назначение и область применения.....	14
3 Проектирование задачи	17
3.1 Обоснование инструментов разработки	17
3.2 Описание алгоритма решения задачи	18
4 Программа решения задачи.....	20
4.1 Логическая структура	20
4.2 Физическая структура.....	21
5 Тестирование и отладка программы	33
5.1 Методика тестирования	33
5.2 Интеграционное тестирование серверной части	33
5.3 Функционально тестирование клиентского приложения	37
6 Применение.....	40
6.1 Назначение программы.....	40
6.2 Требования к аппаратным ресурсам ПК.....	40
6.3 Руководство пользователя.....	41
7 Охрана труда и противопожарная безопасность	46
Заключение	53
Список использованных источников	56
Приложения	57
Приложение А Схема алгоритма	58
Приложение Б Текст программы	61
Приложение В Структура данных.....	65
Приложение Г Скриншоты клиентского приложения	67

Подп. и дата	6.3 Руководство пользователя.....				41
	7 Охрана труда и противопожарная безопасность				46
	Заключение				53
	Список использованных источников				56
	Приложения				57
Взам. инв. №	Приложение А Схема алгоритма				58
	Приложение Б Текст программы				61
	Приложение В Структура данных				65
	Приложение Г Скриншоты клиентского приложения				67
Инв. № дубл.					
Подп. и дата					
Инв. № подл	Лит	Изм.	№ докум.	Подп.	Дата
	Разраб.	Шмелев М.К.			
	Пров.	Обвинникова С.И.			
	Н. контр.	Обвинникова С.И.			
	Утв.	Мирошниченко Г.В.			
ДП-ПР-41-21-2026-ПЗ					
Разработка автоматизированной системы для кафе					
Лит		Лист		Листов	
		8		70	
ЕКТС					

Введение

Информационные технологии сегодня являются ключевым фактором в нашей жизни и на каждом предприятии. Сфера услуг общественного питания также нуждается в автоматизации рутинных процессов, связанных с заказами. Компании ищут лучшие автоматизированные системы из-за высокой конкуренции за клиентов и сотрудников.

В современных условиях кафе должно быть необходимо быстро и качественно обслуживать большой поток клиентов заведения. Ручная обработка заказов с использованием блокнотов приводит к ошибкам: забытые блюда, путаность в заказах, задержка выдачи. Внедрение автоматизированной системы для кафе позволит не только ускорить работу кухни и зала, но и сделает отчетность прозрачной, что способствует повышению выручки.

Актуальность темы подтверждается тем, что каждый день создаются новые организации, которым требуются такие системы. На рынке много предложений: есть как дорогие решения, так и попроще для менее популярных заведений. Всё зависит от требований заказчика. Однако малое или среднее предприятие чаще нуждается в индивидуальных инструментах нежели крупные сети похожие друг на друга. Создание индивидуальной автоматизированной системы для кафе, лишит избыточности функционала и будет дешевле в эксплуатации нежели дорогое ПО.

Многие готовые решения по типу R-keeper или iiko, требуют значительной оплаты их программного обеспечения, а именно: покупка лицензий, подписки с разным функционалом. Так же ПО не в открытом доступе и полноценно настроить под свое предприятие не получится. Эти системы перегружены функционалом, там слишком много всего и не всем подойдут те функции, которые есть там, из этого вытекает еще одна проблема: чтобы содержать ПО, которое превосходит аналоги функционалом, требует современное оборудование. Это значительно увеличивает бюджет, и делает использование данного ПО дорогим. Кроме того, сложность интерфейса требует долгого обучения сотрудников, а любая необходимая уникальная доработка под свое предприятие станет невозможным.

Помимо этого, при использовании таких систем предприятие фактически становится зависимым от поставщика ПО. Если компания прекратит поддержку продукта, поднимет цену или изменит условия лицензии, у заведения не будет выбора, кроме как принять эти условия или полностью менять систему, что само по себе дорого и долго. При разработке собственного решения такой проблемы не возникает, система остаётся в руках заказчика.

Целью дипломного проекта является разработка автоматизированной системы, состоящей из серверной части и клиентского приложения, для автоматизации обработки заказов в кафе.

Результатом данного проекта станет готовая к внедрению автоматизированная система с понятным интерфейсом и понятным сервером, которая предоставит персоналу кафе необходимый функционал для приёмов заказов, контроля заказов на кухне, и финансовый отчетности, не требуя дорогостоящего оборудования.

Инв. № подл	Подп. и дата				
	Взам. инв. №				
	Инв. № дудл.				
	Подп. и дата				
ДП-ПР-41-21-2026-ПЗ					
Ли	Изм.	№ докум.	Подп.	Дата	Лист 10

1 Описание предметной области

Успех современного ресторана или кафе сегодня полностью зависит от того, насколько быстро и четко общаются между собой зал, кухня и руководство. Ресторанный бизнес — это сложный механизм, где блюдо готовится и подается гостю практически в одно и то же время. Поэтому любая заминка или путаница в заказе на любом этапе мгновенно превращается в финансовые потери, портит репутацию заведения, делает гонку выдачи заказов, что путает правила сервиса.

Предметной областью является управление предприятием общественного питания.

В настоящее время, если в заведении нет автоматизированной системы, процесс выглядит так: официант принимает заказ, записывает его в блокнот, затем передает данные кассиру и повару.

Этот подход очень неудобен, если в зале более пятидесяти столов и все они заняты, а официантов всего один или два человека, они физически не успеют качественно обслужить всех гостей. При высокой нагрузке смысл ручного метода пропадает.

Отсутствие контроля провоцирует целый ряд ошибок - блюда часто выносятся не на те столы, что вызывает раздражение гостей и конфликт с персоналом. Много случаев, когда пожелания гостя не учтены к заказу, а если гостей несколько за одним столом, а таких занятых столиков может быть от 20, человек физический не запомнит пожелания гостей и допустит ряд ошибок, которые испортят репутацию предприятия.

Отдельная проблема - нарушение курсов подачи. Официант может вынести десерт раньше основного блюда, что испортит впечатление от вкуса и от вечера. Кроме того, важна одновременная подача: если гости заказали два блюда, недопустимо принести одно блюдо, заставив второго гостя ждать и смотреть, как первый ест. Диаграмма последовательности, изображённая на рисунке 1.1, отображает данный подход.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.						Лист 11
Ли	Изм.	№ докум.	Подп.	Дата	ДП-ПР-41-21-2026-ПЗ				

Также существует риск кражи: сотрудники могут обманывать руководство, присваивая разницу между чеком и полученными деньгами, и от этого будет меньше выручка, это влечет за собой такие последствия: экономия на ресурсах для приготовления блюд, экономия сервиса ресторана, а именно чистота приборов, материалы для сервировки, настольные аксессуары и т.д. В разрабатываемой системе происходит учет денежных средств, что обеспечивает отчетность.

Из этого можно сделать вывод что в XXI веке ни одно среднее и крупное предприятие общественного питания не сможет выдержать конкуренцию и работать без автоматизированной системы.

Диаграмма показывает протекающий процесс актеров без автоматизированной системы: гость заказывает позицию у официанта, оплачивает свой заказ, официант записывает в блокнот, передает данные на кассу, касса печатает чек официанту на оплату. Повар готовит заказ и выдает официанту готовое блюдо, демонстрация на рисунке 1.1.

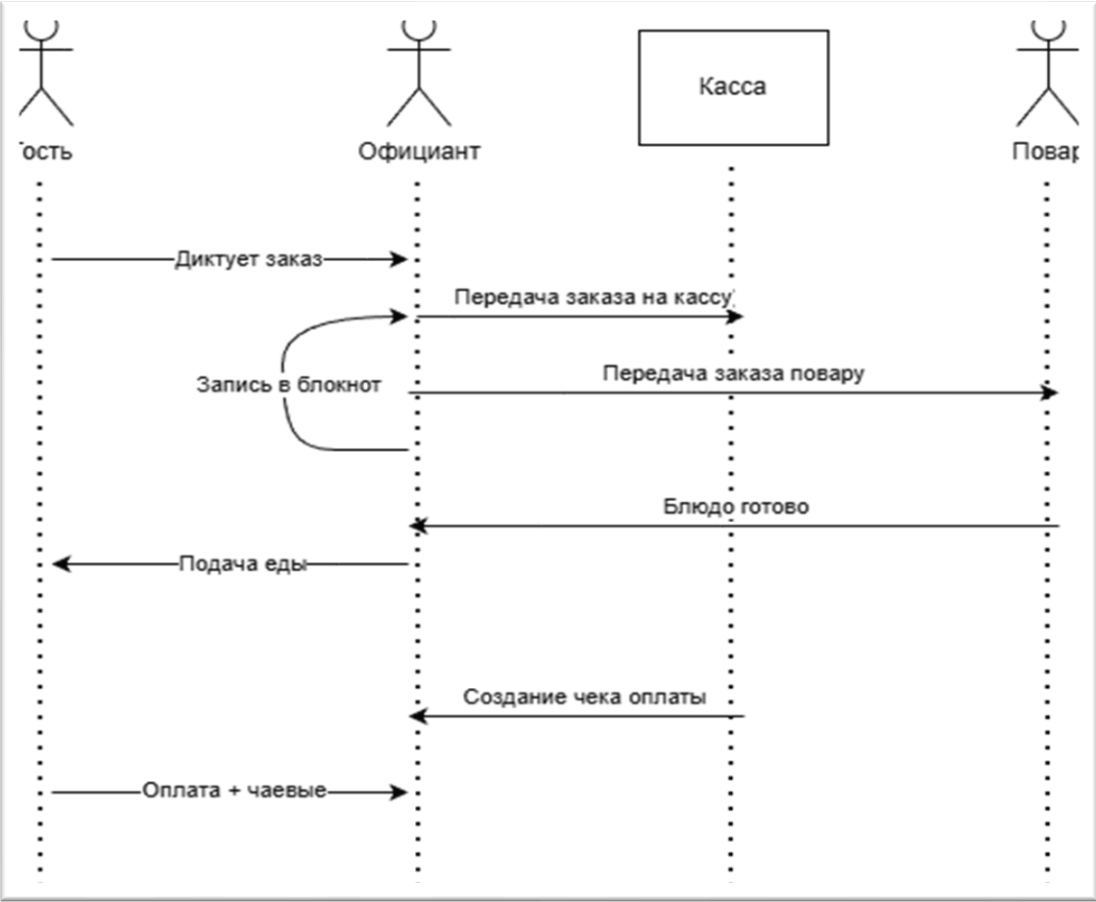


Рисунок 1.1 - Диаграмма последовательности.

Главная проблема заключается в избыточном количестве ручных действий, приводящих к несогласованности. Официант выступает в роли посредника, который тратит время на перемещение в зале, кассой, кухней вместо того, чтобы следить за сервисом в кафе. И если проблема будет на этапе записи в блокнот, то весь процесс идет к сбою цепочки. Для этого и создаются автоматизированные системы кафе/ресторана, чтобы можно было в спешке нормально работать, не совершая ошибок, которые влекут за собой издержки и экономию на сервисе для гостей. Таким образом внедрение специального программного обеспечения позволяет полностью перевернуть физическую цепочку: оно будет исключать человеческий фактор на всех этапах работы кроме приготовления блюд и подачи, и оптимизирует работу среди персонала так как нагрузки станут меньше. В конечном счете автоматизированная система для кафе упрощает и дает свободу от рутинных ручных действий, перенаправляя ресурсы на качество и сервис обслуживания клиентов.

Инв. № подл	Подп. и дата		Взам. инв. №	Инв. № дудл.	Подп. и дата	Инв. № подл	ДП-ПР-41-21-2026-ПЗ					Лист
												13
	Ли	Изм.	№ докум.	Подп.	Дата							

2 Назначение и область применения

Название программного продукта «Автоматизированная информационная система для управления заказами на предприятии общественного питания (кафе)». Новый программный продукт призван комплексно автоматизировать все бизнес-процессы кафе. В его функционал входит ведение электронного меню, маршрутизация заказов между залом и кухней, управление заполненностью столов и визуализация процесса приготовления еды. Приложение станет единой цифровой платформой, объединяющей работу официантов, поваров и администраторов, что позволит минимизировать человеческий фактор и ускорить обслуживание посетителей.

Основные модули выполняющие следующие задачи: сперва модуль авторизации и контроля доступа на основе ролевой модели (Администратор, Официант, Повар); дальше интерактивный каталог блюд с возможностью управления контентом и ценообразованием. Затем модуль заказов и обработки заказов с привязкой к плану столов. Также модуль «Кухонный монитор» для отслеживания очереди готовки в режиме реального времени. И модуль кассового расчета для автоматического формирования итоговой суммы чека.

Функциональные возможности системы: управление учетными записями: регистрация новых сотрудников, безопасная авторизация с помощью JWT-токена, назначение роли каждому сотруднику, также доступ к некоторому функционалу ограничен правами администратора. Управление каталогом меню: добавление новых позиций, редактирование описания, изменения стоимости и категорий блюд, а также добавление фото к блюду. Управление столами: на карте визуализирующую помещение кафе производится мониторинг занятости столов и также выбора стола для создания заказа. Синхронизация с кухней: мгновенная передача заказа на кухонный терминал с приложением под правами повара с деталями заказа. Управление статусами готовности: смена происходит при готовности блюда либо же при закрытии счета заказа. Автоматический расчёт стоимости заказа: итоговый счет считается автоматический при добавлении или удалении позиций. Перевод

отслеживания очереди готовности в режиме реального времени. И модуль кассового расчета для автоматического формирования итоговой суммы чека.

Функциональные возможности системы: управление учетными записями: регистрация новых сотрудников, безопасная авторизация с помощью JWT-токена, назначение роли каждому сотруднику, также доступ к некоторому функционалу ограничен правами администратора. Управление каталогом меню: добавление новых позиций, редактирование описания, изменения стоимости и категорий блюд, а также добавление фото к блюду. Управление столами: на карте визуализирующую помещение кафе производится мониторинг занятости столов и также выбора стола для создания заказа. Синхронизация с кухней: мгновенная передача заказа на кухонный терминал с приложением под правами повара с деталями заказа. Управление статусами готовности: смена происходит при готовности блюда либо же при закрытии счета заказа. Автоматический расчет стоимости заказа: итоговый счет считается автоматический при добавлении или удалении позиций. Перевод

заказов в архив: создано это для безопасности и учета денежных средств во время оплаты чтобы не поменять стоимость после оплаты заказа.

Сравнение программного обеспечения с его аналогами: на табл. 2.1 проведено сравнение аналогов.

Разрабатываемая система - менее нагруженная, бесплатная, не требует мощного ПО, можно обойтись без POS-терминалов. Запутаться в функциях не получится, есть базовые и готовые решения для обработки заказов. Это экономит время обучение персонала, сложность минимальная. Низкие системные требования - этот продукт можно развернуть на любом офисном ПК. И заказчик получает полный контроль над сервером, что в будущем играет большую роль в расширении и масштабировании.

ПКО - предоставление комплексной автоматизированной системы, решение для всех типов заведений. Охватывает все аспекты бизнеса – от кассы до склада. Недостаток в том, что имеет высокую стоимость для малого бизнеса от 10 - 32 тыс. руб. При этом платить такие деньги нужно каждый месяц за подписку, что дорого для начинающего бизнеса. Помимо этого, ПКО требует мощное настольное обеспечение, а именно: ПК, POS-Терминалы. Система сама по себе сложная, придется долго обучать персонал.

R-Keeper - для сложных и очень крупных сетей. Архитектура и зависимость к оборудованию, плюсом нужен обученный человек чтобы постоянно следить за системой. Огромное количество разных модулей продаж. Дорогая система для покупки полной версии приложения как от 60 - 150 тыс. руб. Эта сумма платиться за лицензию. Из-за сложной архитектуры программа отказывается работать на офисных ПК, оно требует профессиональное дорогое мощное оборудование, чтобы запускать такую тяжелую автоматизированную систему. В интерфейс очень много разных модулей которые не всем пригодятся, обучение персонала производит IT-специалист, который это устанавливает.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	Инв. № подл.						ДП-ПР-41-21-2026-ПЗ	Лист 15
						Ли	Изм.	№ докум.	Подп.	Дата		

Таблица 2.1 - сравнение лидеров рынка с разрабатываемой системой

<i>Продукты</i>	<i>ИКО</i>	<i>R-Keeper</i>	<i>Разрабатываемая система</i>
аудитория	Все типы заведений	крупные ресторанные сети.	Малый и средний бизнес.
Стоимость внедрения	Средняя варьируется от 10.000 – 32.000 тыс. Руб. еще ежемесячная подписка	Высокая (60 000 – 150 000 руб. за полную версию).	Бесплатная (затраты только на разработку и внедрение).
Оборудование	Требуются POS-терминалы и совместимое оборудование.	Высокие требования к ПК и POS - Терминалам	Низкие требования. Работает на любом офисном ПК,
Сложность обучение	Охватывает все аспекты (от кассы до склада), требует обучения.	Высокая сложность. Очень много модулей ведения учета.	Минимальная сложность. Быстрое обучения персонала.
Обслуживание	Требует техподдержки.	Требуется обученный специалист.	Простое. Не требует обученного специалиста.
Функциональность	Комплексная (Касса, Склад, Кухня, Финансы, Персонал).	Мощная система с разными вариантами продаж.	Готовое решения для обработки заказов, ничего лишнего.

Сравнение показывает, что автоматизированная система хорошая альтернатива лидерам рынка. ИКО и R-Keeper созданы для гигантов, стоят дорого и требуют мощного оборудования. Мой продукт решает проблему малого бизнеса, а именно такие задачи: бесплатный, запускается на офисном ПК, простой, интерфейс и функционал не перегружен. Персонал научится работать буквально за один день. Моя система это выгодное решение без больших трат денежных средств.

Инв. № подл.	Подп. и дата
Взам. инв. №	
Инв. № дубл.	
Подп. и дата	
Инв. № подл.	

Ли	Изм.	№ докум.	Подп.	Дата	ДП-ПР-41-21-2026-ПЗ	Лист
						16

3 Проектирование задачи

3.1 Обоснование инструментов разработки

Проектирование важная часть в разработке программного обеспечения (ПО). Выбор технологий для реализации системы выбран для быстрой работы обработки данных и стабильности данной системы.

Для серверной части был выбран фреймворк ASP.NET Core API. Основным аргументом послужила его высокая производительность и простота в сфере настройки web-части, а именно обработка асинхронных запросов, быстрая работа контроллеров и много способов работы с БД.

В качестве СУБД сравнивались реляционный БД, а именно MS SQL Server и PostgreSQL. Несмотря на бесплатность и открытый исходный код PostgreSQL, выбор был сделан в пользу MSSQL из-за ее бесшовной связи с EFCore в .Net. Это существенно упрощает написание кода, администрирования БД, максимально быстро генеративные SQL-запросы от расширения идут базу данных, и показывает высокую целостность передачи данных.

Безопасность системы для обеспечения доступов реализована через JWT токены. Роли и данные пользователя вшиваются в токен что позволяет управлять доступом. Так же пароли хранятся в базе данных в виде хэша, выполненным алгоритмом от библиотеки BCrypt.

Для обеспечения удобства взаимодействия между серверной и клиентской частями системы, а также для упрощения процесса отладки, внедрена автоматическая документация на базе инструмента Swagger.

Развертывание и облегчение переносимости было реализовано Docker. Позволяет упаковать серверную часть с базой данных в изолирующую среду (контейнер) что гарантирует абсолютно одинаковую работу.

Клиентское приложение разработано на WPF. В отличие от веб-решений, десктопное приложение на WPF работает быстрее в условиях локальной сети кафе и позволяет использовать библиотеку `MaterialDesignInXamlToolkit` для создания

красивого интерфейса. Взаимодействие с API происходит через класс HttpClient, а парсинг данных с помощью System.Text.Json.

3.2 Описание алгоритма решения задачи

UML - диаграммы: диаграмма последовательности показывает, как протекает бизнес-процесс во время его работы, как работают сотрудники, система, представлено на рисунке 3.1 На диаграмме изображена последовательность действий при выборе автоматизированной системы для обработки заказов.

Клиент делает заказ у официанта, при необходимости может изменить выбранные позиции и затем производит оплату. Официант принимает заказ и вносит его в систему. Повару предоставляется интерфейс для просмотра поступивших заказов и их статусов. WPF-приложение выступает связующим звеном между персоналом и сервером: все действия сотрудников передаются на сервер через приложение. Серверное API содержит основную бизнес-логику и отвечает за обработку заказов.

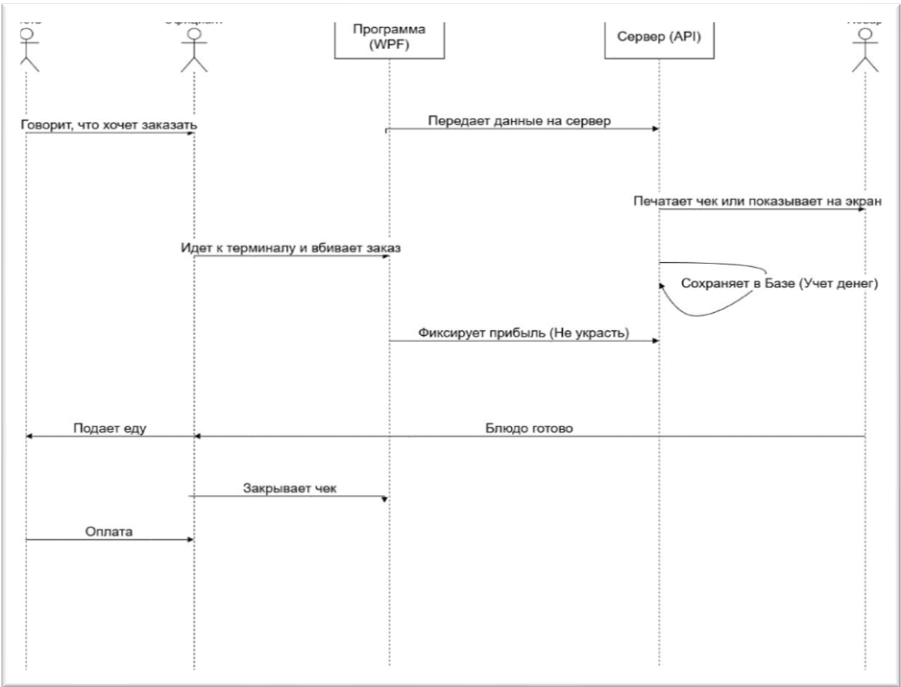


Рисунок 3.1 - Диаграмма последовательности с использованием системы
Процесс взаимодействия можно описать таким: официант принимает заказ у клиента на стойке с POS - терминалом или же блокнотом, подходит к стойке,

заводит заказ в систему в клиентском приложении, дальше данные валидируются и отправляются по API на обработку на сервер с ними происходит прописанная логика, различные проверки и сервер возвращает ответ, дальше заказ переходит на кухню к поварам, когда приготовят официант отмечает в клиент приложении, что заказ готов, заказ уходит на сервер и отмечается как готов и официант его приносит. Дополнительно система обеспечивает синхронизацию статусов заказов между клиентским приложением и серверной частью в реальном времени.

На рисунке 3.2 изображена диаграмма вариантов использования (прецедентов). Диаграмма показывает актеров, которые оперируют с автоматизированной системой. Каждый актер делает свое дело, официант и повар вместе могут смотреть товары, менеджер управляет меню, учетными записями. Это диаграмма показывает процесс взаимодействия прецедентов.

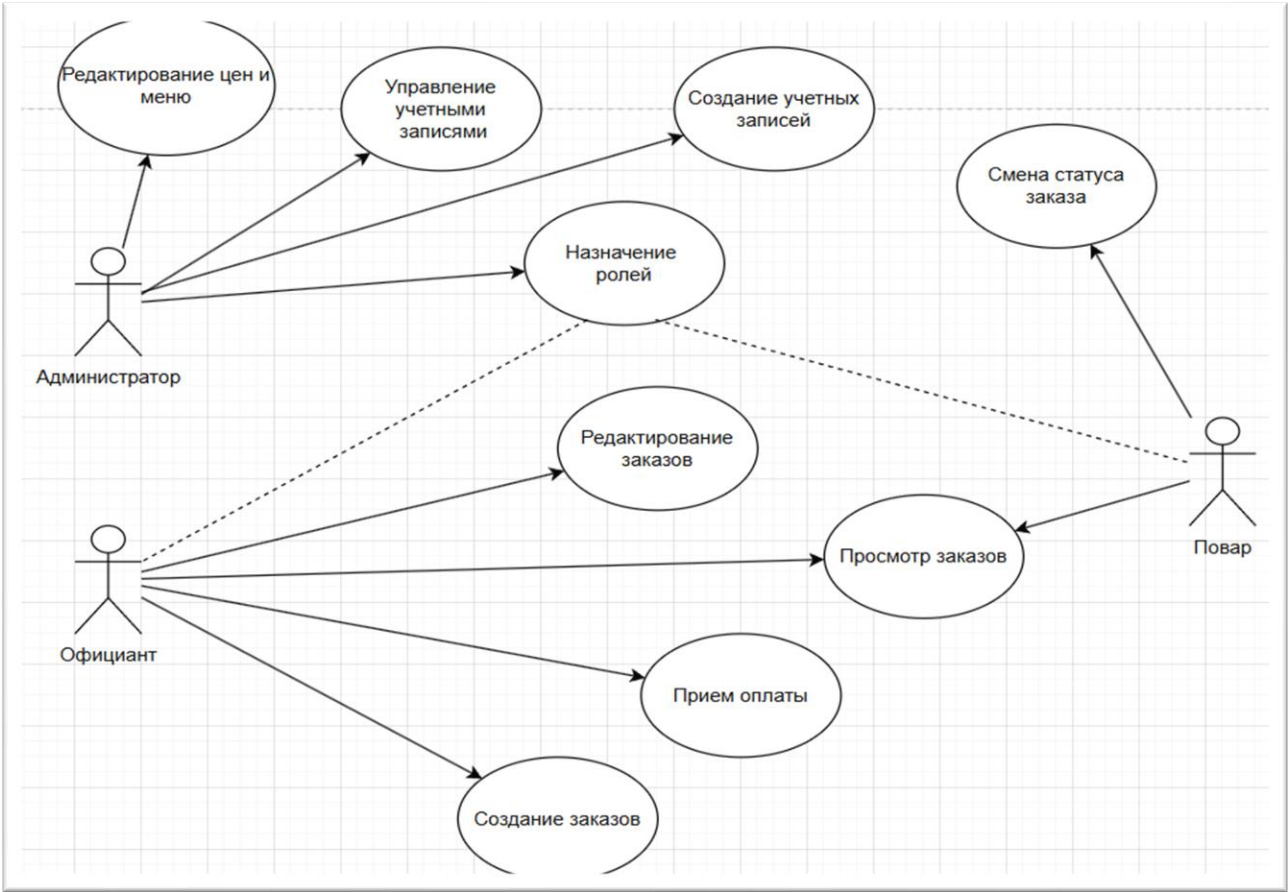


Рисунок 3.2 - Диаграмма прецедентов

4 Программа решения задачи

4.1 Логическая структура

Логическая структура программного продукта базируется на реляционной модели данных. Главная задача была сделать правильную структуру, следуя современным практикам, чтобы обеспечить модульность и возможность масштабирования. Для понимания представлена ER Диаграмма базы данных, изображенная на рисунке 4.1. Которая служит основой для проектирования базовых классов сущностей. Она показывает название полей, связи сущностей, ключи, типы данных. Это очень важный пункт при проектировании логической структуры.

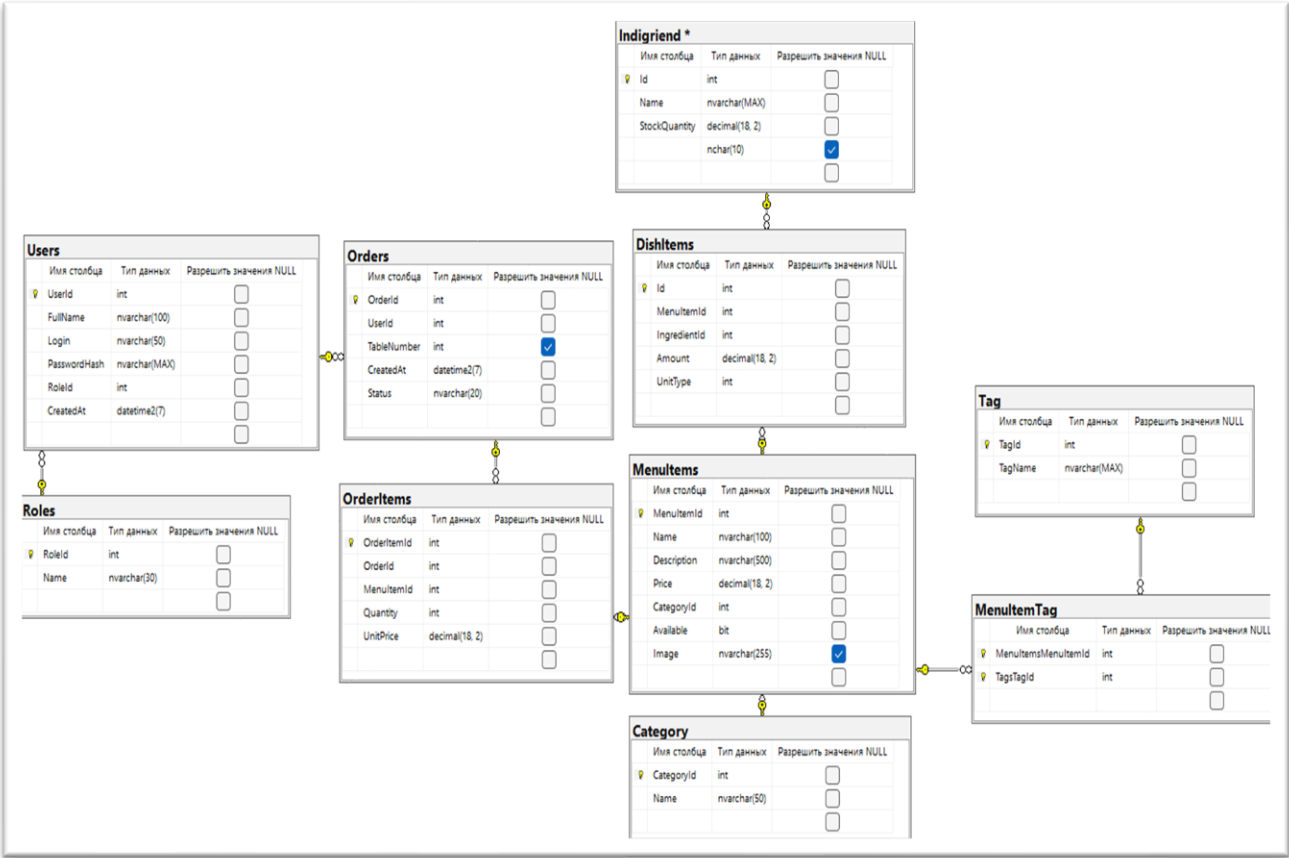


Рисунок 4.1 - ER Диаграмма

Roles (Роли): содержит уровни доступа в системе (Администратор, Официант, Повар).

Users (Пользователи): хранит персональные данные сотрудников, их логины и хэши паролей для безопасной аутентификации.

Подп. и дата
Взам. инв. №
Инв. № дубл.
Подп. и дата
Инв. № подл.

Repositories - отдельный слой, который работает с базой. Предоставляет базовые методы CRUD (создание, чтение, обновление, удаление) для работы с моделями

Models - модели, которые напрямую связаны с таблицами базы данных.

DTOs - объекты для передачи данных между клиентом и сервером. Разделены по папкам (MenuItems, OrderItems, Orders, Roles, Users, Category, Kitchen). Позволяют скрыть структуры бд от клиента.

Data (CafeDbContext) – контекст базы данных, отвечает за соединение с СУБД.

В качестве выбранной базы данных будет использоваться MS SQL и будет управляться через SQL SERVER. Взаимодействие с БД будет использована библиотека EF-Core (Entity Framework). Классы, размещённые в папке Models, соответствуют сущностям базы данных. Миграции базы данных хранятся в каталоге Migrations.

Чтобы понимать, как работает система, а именно паттерн, ниже представлена схема на рисунке 4.2. На изображении файловое дерево проекта, показано какой паттерн проектирования использован и в какой последовательности взаимодействует каждый модуль паттерна.

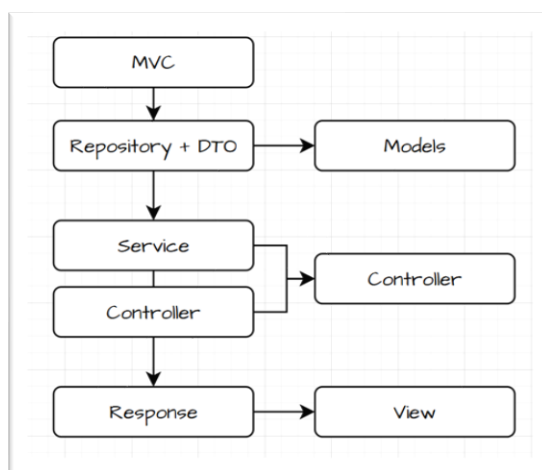


Рисунок 4.2 - Файловое дерево проекта

Разрабатывался программный продукт с использованием многоуровневой архитектуры (N-Layer), что позволяет разделить ответственность между компонентами системы. Реализация серверной части (ASP.NET Core Web API)

Инв. № инв.	Подп. и дата				
Инв. № докл.	Подп. и дата				
Инв. № подл.	Подп. и дата				
Ли	Изм.	№ докум.	Подп.	Дата	Лист
ДП-ПР-41-21-2026-ПЗ					22

построена на основе трех слоев, а именно: Web Layer (Controllers) – принимает HTTP-запросы. Service Layer (Services) – содержит бизнес-логику. Data Access Layer (Repositories) – взаимодействует с базой данных через Entity Framework Core.

Полный исходный код программного продукта размещён в репозитории проекта и приведён в приложении А.

На рисунке 4.3 показано то, как в файле Program.cs настроены внедрение зависимостей (DI - Dependency Injection) Что связывает интерфейсы и их реализацию.

```
builder.Services.AddScoped<IMenuItemRepository, MenuItemRepository>();
builder.Services.AddScoped<IUserRepository, UserRepository>();
builder.Services.AddScoped<IOrderRepository, OrderRepository>();
builder.Services.AddScoped<IOrderItemRepository, OrderItemRepository>();

builder.Services.AddScoped<IUserService, UserService>();
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddScoped<ITokenService, TokenService>();
```

Рисунок 4.3 - Внедрение зависимостей

Модуль аутентификации и безопасности реализован с использованием JWT-токенов. Сервис TokenService отвечает за генерацию токена, содержащего идентификатор пользователя, логин и роль. На рисунке 4.4 изображена реализация данного механизма.

```
var claims = new List<Claim>
{
    new Claim(ClaimTypes.NameIdentifier, user.UserId.ToString()),
    new Claim(ClaimTypes.Name, user.Login),
    new Claim(ClaimTypes.Role, user.RoleId.ToString())
};
```

Рисунок 4.4 - Данные пользователя в токене

Одной из основных функций системы является обработка заказов. Логика данного модуля реализована в сервисе OrderService. Метод создания заказа включает проверку корректности выбранных блюд, расчёт общей стоимости и

Подп. и дата	
Взам. инв. №	
Инв. № дубл.	
Подп. и дата	
Инв. № подл.	

Ли	Изм.	№ докум.	Подп.	Дата

сохранение данных в базе данных. На рисунке 4.5 изображена реализация механизма обработки заказов.

```
public async Task<OrderResponseDto> CreateOrderAsync(CreateOrderDto orderDto)
{
    //Создаем новый заказ
    var order = new Order
    {
        UserId = orderDto.UserId,
        TableNumber = orderDto.TableNumber,
        Status = orderDto.Status,
        CreatedAt = DateTime.UtcNow
    };

    //Добавляем блюда в заказ
    foreach (var item in orderDto.Items)
    {
        var menuItem = await _menuItemRepository.GetMenuItemByIdAsync(item.MenuItemId);
        if(menuItem == null || menuItem.Available == false)
        {
            throw new Exception($"Блюдо {item.MenuItemId} не найдено. или же в стоп листе");
        }
        var newOrderItem = new OrderItem
        {
            UnitPrice = menuItem.Price,
            MenuItemId = item.MenuItemId,
            Quantity = item.Quantity,
            MenuItem = menuItem
        };
        order.OrderItems.Add(newOrderItem);
    }
    await _orderRepository.CreateOrderAsync(order);
    // Формируем ответ
    var orderResponse = new OrderResponseDto
    {
        OrderId = order.OrderId,
        UserId = order.UserId,
        TableNumber = order.TableNumber,
        CreatedAt = order.CreatedAt,
        Status = order.Status,
        TotalAmount = order.OrderItems.Sum(oi => oi.UnitPrice * oi.Quantity),

        Items = order.OrderItems.Select(oi => new OrderItemDto
        {
            OrderItemId = oi.OrderItemId,
            OrderId = oi.OrderId,
            MenuItemId = oi.MenuItemId,
            Quantity = oi.Quantity,
            UnitPrice = oi.UnitPrice,
            MenuItemName = oi.MenuItem?.Name ?? "Неизвестно"
        }).ToList()
    };
    return orderResponse;
}
```

Рисунок 4.5 - Создание заказа

Для документирования API - конечных точек для клиента будет использоваться Swagger. У HTTP есть свои методы: GET - получить, POST - создать, PUT - обновить, DELETE - удалить, PATCH - частичное обновление.

Подп. и дата	
Взам. инв. №	
Инв. № дудл.	
Подп. и дата	
Инв. № подл.	

Ниже документирование конечных точек API с использованием инструмента Swagger. Дополнительно ссылка на опубликованную спецификацию API и QR-код приведены в приложении Б

Menu - здесь конечные точки menu, а именно получение (GetMenu), добавление (Add), обновление (Update), удаление (DeleteItem), получение по id (MenuItemId), добавление количества продуктов, прибывшие на предприятие (CreateProduct), добавление и изменение фото блюда (image), получение категорий (GetCategories). Права администратора требуются для любых изменений (Add, Update, Delete), в то время как официанты имеют доступ только к чтению информации. Взаимодействие происходит через DTO: MenuItemResponseDto (фильтрация выходных данных), CreateMenuItemDto (макет для создания) и UpdateMenuItemDto (ограниченное редактирование полей). Конечные точки модуля представлены на рисунке 4.6.

Menu	
GET	/api/Menu/GetMenu
POST	/api/Menu/Add
DELETE	/api/Menu/{id}
GET	/api/Menu/{id}
POST	/api/Menu/CreateProduct
POST	/api/Menu/{id}/image
GET	/api/Menu/GetCategories

Рисунок 4.6 - Конечные точки Menu

Orders - Эндпойнты модуля orders выполняют следующие операции. Создание заказа (CreateOrder), получение всех заказов (GetAll), получение по id (GetOrderById), добавить позиции в заказ (AddItemsToOrder), удалить заказ (DeleteOrder), получение заказов, закрепленных за конкретным пользователем (userOrder), получить статус заказа (status), обновить статус заказа (statusUpdate), и удалить позицию из заказа (deleteItem), получение активных заказов для экрана кухни (GetActiveOrders), также реализован экспорт данных о выручке в формат Excel (ExportRevenue) за день или месяц. Права администратора требуются только для Excel отчета, в то время как официанты имеют полный доступ.

Для запросов по адресу Orders используется DTO OrderResponseDto - возвращает полный ответ сервера после создания или запроса заказа, содержит статус, дату создания и вложенный список блюд, скрывая служебную логику. CreateOrderDto - реализует макет данных при создании нового заказа, требует указать сотрудника, номер стола и список позиций. OrderItemDto - используется как вложенный объект, описывает конкретное блюдо и его количество в составе заказа. Конечные точки модуля представлены на рисунке 4.7.

Orders	
POST	/api/Orders/CreateOrder
GET	/api/Orders/GetAll
GET	/api/Orders/{id}/GetOrderById
POST	/api/Orders/{id}/AddItemsToOrder
DELETE	/api/Orders/{id}/DeleteOrder
GET	/api/Orders/{userId}/userOrder
GET	/api/Orders/{status}/status
PATCH	/api/Orders/{id}/statusUpdate
DELETE	/api/Orders/{id}/deleteItem
GET	/api/Orders/GetActiveOrders
GET	/api/Orders/ExportRevenue

Рисунок 4.7 - Конечные точки

User - модуль, содержащий конечные точки для управления пользователями системы: вход в приложение (Login), регистрация (Register), получение списка пользователей (User), получение пользователя по идентификатору (GetUserId), обновление данных (UpdateUser) и удаление пользователя (DeleteUser). Для передачи данных между клиентом и сервером используются DTO. Доступ к маршруту (api/User), а также операциям DeleteUser и GetUserId разрешён только

пользователям с правами администратора, Конечные точки модуля представлены на рисунке 4.8.

User			^
POST	/api/User/Login		🔒 ▼
GET	/api/User		🔒 ▼
POST	/api/User/Register		🔒 ▼
DELETE	/api/User/DeleteUser		🔒 ▼
GET	/api/User/{id}/GetUserId		🔒 ▼
PATCH	/api/User/{id}/UpdateUser		🔒 ▼

Рисунок 4.8 - Конечные точки User

Серверная часть приложения разворачивается в Docker-контейнере. Для обеспечения переносимости, изоляции, упрощения процесса использовалась контейнеризация Docker. Это гарантирует одинаковое поведение приложения как на локальной машине разработчика, так и на сервере. Конфигурация контейнера описана в Dockerfile, представленном на рисунке 4.9.

```
# Образ для запуска сервера
FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS base
USER $APP_UID
WORKDIR /app
EXPOSE 8080
EXPOSE 8081

# Образ для сборки приложения
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
ARG BUILD_CONFIGURATION=Release
WORKDIR /src
COPY ["CafeAPI/CafeAPI.csproj", "CafeAPI/"]
RUN dotnet restore "./CafeAPI/CafeAPI.csproj"
COPY . .
WORKDIR "/src/CafeAPI"
RUN dotnet build "./CafeAPI.csproj" -c $BUILD_CONFIGURATION -o /app/build

# Сборка и публикация приложения
FROM build AS publish
ARG BUILD_CONFIGURATION=Release
RUN dotnet publish "./CafeAPI.csproj" -c $BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false

# создание исполняемого образа
FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
USER root
RUN mkdir -p /app/wwwroot/images && chown -R $APP_UID /app/wwwroot
USER $APP_UID
ENTRYPOINT ["dotnet", "CafeAPI.dll"]
```

Рисунок 4.9 - Dockerfile

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата



Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата



Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

И.В. № подл.	Подп. и дата	И.В. № докл.	Взам. инв. №	Подп. и дата

Рисунок 4.15 - Добавления товара

BCrypt.Net-Next - безопасное хранение паролей C# (и .NET), это библиотека используется только для паролей она не шифрует отдельные данные, она медленная по сравнению с аналогами, но проста в использовании для системы, использует криптографическую хеш-функцию.

ДП-ПР-41-21-2026-ПЗ

.NET 9 и asp.net core, потому что они идеально подходят к созданию API, фреймворк очень удобен тем, что выполняет много автоматических задач, например он маршрутизирует запросы, какой контроллер обрабатывает HTTP запрос (GET, POST, PUT, DELETE). Преобразует объекты в форматы JSON для отправки клиенту и обратно десериализует. Управление жизненным циклом с помощью middleware – конвейером. И предоставляет готовые ресурсы для создания точек.

MS SQL - реляционная БД, хорошо работает с .NET, несложная, можно управлять через интерфейс в программе SQL Server, через визуал проще взаимодействовать с данными и управлять ими, так же возможно подключить к Visual Studio и управлять ее данными через IDE. Паттерн я выбрал чтобы каждый слой не знал друг о друге, система немаленькая и так проще будет тестировать каждый блок и чинить в случае поломки.

Аналоги: Windows Presentation Foundation указаны в списке ниже.

WPF - мощный фреймворк для десктопных приложений, работает с .NET на чем и работает автоматизированная система, оптимизирована под API. По сравнению с аналогами он гарантированно хороший выбор. UWP - Universal Windows Platform устаревшая система, на замену ему пришел WinUI 3, он менее гибкий по сравнению с WPF или WinUI. Avalonia UI - мощный фреймворк для создания десктопных приложений на windows похож на WPF, но он является сторонним решением (не от Microsoft).

Ниже приведено детальное описание каждой страницы клиентского приложения. Скриншоты страниц находятся в Приложение Г. Страница авторизации (LoginPage) для входа в систему. Содержит поля для ввода логина и пароля, после заполнения данные отправляются на сервер. В ответ система отдает JWT-токен. На его основе создается объект, отвечающий за хранение информации о пользователе и выдачи прав доступа в зависимости от роли.

Главная страница (MainPage). Этот экран выполняет роль основного пульта управления для персонала. Интерфейс разделен на 3 раздела.

Инв. № подл	Подп. и дата	Инв. № докл.	Взам. инв. №	Подп. и дата						Лист 31
Ли	Изм.	№ докум.	Подп.	Дата	ДП-ПР-41-21-2026-ПЗ					

Список активных и архивных заказов, поле результатов поиска, категории для быстрого поиска блюд. Так же сверху есть меню, видимость некоторого функционала только под правами администратора. Создание заказа, обновить, управление меню и пользователями, так же отчет о выручке, выход из системы.

Создание нового заказа (CreateOrderPage). Данный экран визуализирует помещение предприятия, которое использует автоматизированную систему, на карте изображены столы. Занят ли стол или нет это отображается на карте, а именно сменой цвета стола и блокировкой выбора.

Детализация заказа (OrderDetailsPage). Данный экран используется для управления выбранным заказом. Здесь отображаются все блюда с указанием количества и стоимости. Официант может удалять или добавлять новые позиции, так же есть функция разделение счета на n-количество в зависимости от блюд. Конечный этап — это подсчет итоговой стоимости заказа и функция Закрытия счета после который заказ становится оплаченным и переходит в архив.

Карточка блюда (DishPage). Демонстрирует расширенную информацию выбранной позиции из меню. Выводит информацию, именно: Изображение блюда, название, описание, категорию, цену и так же состав блюда. С правами администратора можно назначить фото для данного блюда.

Управление меню (MenuPage). Административная страница для управления каталогом предприятия. Позволяет добавлять блюда, удалять.

Экран кухни (KitchenPage). Страница для поваров, отображающая список заказов в реальном времени, каждый заказ — это отдельная карточка с подробностями, когда блюдо готово нужно нажать на статус заказа, после этого заказа сменит статус на готовый.

Управление пользователями (UserPage). Этот экран доступен только администратору и предназначен для управления персонала. Он позволяет добавлять новых сотрудников в систему, так же изменять их данные

Инв. № подл	Подп. и дата	Взам. инв. №	Инв. № дубл.						
Инв. № подл	Подп. и дата	Взам. инв. №	Инв. № дубл.						
Ли	Изм.	№ докум.	Подп.	Дата	ДП-ПР-41-21-2026-ПЗ				Лист
									32

5 Тестирование и отладка программы

5.1 Методика тестирования

Тестирование программного продукта проводилось с целью проверки корректности работы реализованных модулей, устойчивости системы к ошибочным действиям пользователя, а также соответствия функциональных возможностей заявленным требованиям. Для обеспечения высокой надежности системы было решено разделить процесс на два уровня:

Первое это автоматизированное интеграционное тестирование серверной части (API). Проверка взаимодействия контроллеров с базой данных, маршрутизация и функций авторизации. А второе функциональное тестирование клиентского приложения (WPF). Проверка отзывчивости интерфейса через принцип “Черного ящика”.

5.2 Интеграционное тестирование серверной части

Для проверки устойчивости серверной части (ASP.NET Core Web API) было разработано 14 интеграционных авто тестов с использованием NUnit и инструмента построения отчетов Allure. Инфраструктура тестов находится в классе CafeApiFactory он наследуется от WebApplicationFactory<Program>. Этот подход позволяет разворачивать тестовый сервер в оперативной памяти, полностью изолируя и имитируя настоящую среду выполнения HTTP запросов. Для независимости тестов от друг друга было принято решение сделать класс BaseIntegrationTest. Перед каждым запуском тестов происходит чистка и пересоздание контекста базы данных что исключает влияние не понятных данных на результаты тестов. Так же в базовом классе происходит генерация JWT-токена для эндпоинтов которые защищены аннотацией [Authorize].

Автоматизированным тестирование покрыты все контроллеры, которые есть на сервере, а именно: сначала MenuControllerTests: проверка добавления, получения

и удаления позиций меню с проверкой сохранения в базу данных. Сделано 4 готовых теста.

Затем OrdersContollerTests: проверка жизненного цикла заказа. Выполняются такие сценарии как создание заказа, добавление позиций, проверки привязанности к конкретному пользователю и обновление статусов заказов. Сделано 8 готовых тестов. Такие сценарии очень важны, потому что они полностью имитируют работу с модулем заказов, авто тесты контролируют чтобы все работало корректно при любой ситуации.

UserControllerTests: проверка безопасности, регистрация и аутентификация. Сделано 2 готовых теста. Ниже представлен рисунок 5.1 на котором фрагмент кода интеграционного теста, проверяющего процесс регистрации нового сотрудника под правами администратора.

```
[Test]
Bikmacs
public async Task RegisterUser_AsAdmin_Success()
{
    AuthenticateAdminAsRole();
    var response = await HttpClient.PostAsJsonAsync( requestUri: "/api/User/Register", _user);
    var responseString = await response.Content.ReadAsStringAsync();

    Assert.That(response.IsSuccessStatusCode, Is.True,
        message: $"Сервер не смог зарегистрировать пользователя. Ошибка: {responseString}");
    Dbcontext.ChangeTracker.Clear();

    var userInDb = Dbcontext.Users.FirstOrDefault(x :User => x.Login == _user.Login);

    Assert.That(userInDb, Is.Not.Null, message: "Пользователь не был сохранен в базу данных");
    Assert.That(userInDb.Login, expression: Is.EqualTo(_user.Login), message: "Логин сохранился не так");
    Assert.That(userInDb.RoleId, expression: Is.EqualTo(_user.RoleId), message: "Роль назначена неверно");
    Assert.That(userInDb.PasswordHash, expression: Is.Not.EqualTo(_user.Password),
        message: "ОПАСНОСТЬ: Пароль сохранен в открытом виде без хэширования");
}
```

Рисунок 5.1 - Фрагмент интеграционного теста регистрации пользователя

Так же еще пример интеграционного теста, проверяющий добавление новых позиций в уже существующий заказ, изображен на рисунке 5.2

Ид. № подл.	Подп. и дата	Взам. инв. №	Ид. № докл.	Подп. и дата	Ид. № подл.	Лист
						34
Ли	Изм.	№ докум.	Подп.	Дата	ДП-ПР-41-21-2026-ПЗ	

Тесты проверяют и верифицируют статус коды от сервера (через Assert.That функция сравнения ожидания и реальности) и непосредственно проверяют базу данных на наличие добавленного блюда в данный заказ.

```
[Test]
[Bikmacs]
public async Task AddItemsToOrder_ValidItems_AddsSuccessfully()
{
    AuthenticateWaiterAsRole();

    var existingOrder = new Order { UserId = 1, TableNumber = 1, Status = "Готовится" };
    Dbcontext.Orders.Add(existingOrder);

    Dbcontext.MenuItems.Add(_testMenuItem);
    await Dbcontext.SaveChangesAsync();

    var requestDto = new CreateOrderDto
    {
        UserId = 1,
        TableNumber = 1,
        Status = "Готовится",
        Items = [new OrderItemCreateDto { MenuItemId = _testMenuItem.MenuItemId, Quantity = 2 }]
    };

    var response = await HttpClient.PostAsJsonAsync(requestUrl: $"/api/Orders/{existingOrder.OrderId}/AddItemsToOrder", requestDto);
    Assert.That(response.StatusCode, expression: Is.EqualTo( expected: HttpStatusCode.OK));
    var responseBodyString = await response.Content.ReadAsStringAsync();
    Assert.That(response.StatusCode, expression: Is.EqualTo( expected: HttpStatusCode.OK), message: $"Ошибка: {responseBodyString}");
    Dbcontext.ChangeTracker.Clear();

    var orderItem = Dbcontext.OrderItems.FirstOrDefault(o => o.OrderId == existingOrder.OrderId && o.MenuItemId == _testMenuItem.MenuItemId);

    Assert.That(orderItem, Is.NotNull, message: $"Товар не добавился в базу к заказу");
    Assert.That(orderItem.Quantity, expression: Is.EqualTo( expected: 2));
}
```

Рисунок 5.2 - Фрагмент интеграционного теста добавления в заказ

Конечный тест, который проверяет работоспособность добавления позиции в меню. Он создает таблицу, добавляет тестовый объект и проводит проверку есть ли он в базе данных, фрагмент кода на рисунке 5.3.

```
[Test]
[Bikmacs]
public async Task AddEatOnMenu()
{
    Dbcontext.Category.Add(_testMenuItem.Category);
    await Dbcontext.SaveChangesAsync();

    var testMenuItem = new CreateMenuItemDto
    {
        Name = "Чизкейк",
        Description = "Вкусный чизкейк",
        Price = 350,
        CategoryId = _testMenuItem.Category.CategoryId,
        Available = true,
    };

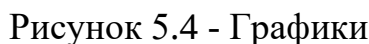
    var response = await HttpClient.PostAsJsonAsync(requestUrl: "/api/Menu/Add", testMenuItem);
    Assert.IsTrue(response.IsSuccessStatusCode, message: $"Сервер вернул: {response.StatusCode}");

    var itemDb = Dbcontext.MenuItems.FirstOrDefault(t => t.Name == "Чизкейк");

    Assert.IsNotNull(itemDb, message: "Блюдо не было сохранено в базу данных!");
    Assert.AreEqual(350, itemDb.Price, message: "Цена сохранилась неправильно!");
}
```

Рисунок 5.3 - Фрагмент интеграционного теста добавления в меню

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата



Пакеты

порядок | имя | длительность | статус | Статус: ❌ ⚠️ ✅ 🔄 💡

Методы: 👁 🔍 📄 🗨 🔊

- IntegrationTests.Controllers
 - MenuControllerTests
 - #1 AddEatOnMenu 3s 397ms
 - #2 DeleteMenuAsync_ReturnMenu 224ms
 - #3 GetMenuAsync_ReturnMenu 167ms
 - #4 GetMenuItemById 115ms
 - OrdersControllerTest
 - #1 AddItemsToOrder_ValidItems_AddsSuccessfully 767ms
 - #2 CreateOrderTest 193ms
 - #3 DeleteItemsToOrder_ValidItems_sSuccessfully 144ms
 - #4 DeleteOrderItem_ValidData 134ms
 - #5 GetAllOrdersTest 271ms
 - #6 GetUserOrder_ValidUserId_ReturnsOKWithOrder 115ms
 - #7 ResponseStatusOrder_ValidStatus_ReturnsOKWithOrder 123ms
 - #8 UpdateStatusOrder_ValidStatus 140ms
 - UserControllerTests
 - #1 LoginUser_Success 1s 876ms
 - #2 RegisterUser AsAdmin Success 955ms

Прошло DeletetemsToOrder_ValidItems_sSuccessfully

Обзор История Параллелизм

Важность: обычная
Длительность: 144ms

Выполнение

▼ Тело теста

```

@ Corssie Output
warn: Microsoft.AspNetCore.StaticFiles.StaticFileMiddleware[16]
The webrootPath was not found: C:\Users\CF\Documents\Cafesystem\Cafesolution\CafeAPI\webroot. Static files may be unavailable.
Info: Microsoft.EntityFrameworkCore.Update[10100]
Saved 500 entities to in-memory store.
Info: Microsoft.EntityFrameworkCore.Update[10100]
Saved 1 entities to in-memory store.
Info: Microsoft.EntityFrameworkCore.Update[10100]
Saved 1 entities to in-memory store.
```

Рисунок 5.5 - Пакеты

HTTP-коды, которые отдает сервер на запрос и соответствующим образом клиентское приложение и тесты реагируют на результат выполнения запроса. Основные коды ответов сервера приведены в табл. 5.6.

Таблица 5.6 - Таблица ответов сервера с расшифровкой кодов.

<i>Код</i>	<i>Название</i>	<i>Описание</i>
200	OK	Запрос успешно выполнен
201	Created	Ресурс успешно создан
400	Bad Request	Ошибка валидации входных данных
401	Unauthorized	Отсутствует или некорректен токен авторизации
403	Forbidden	Недостаточно прав для выполнения операции
404	Not Found	Запрошенный ресурс не найден
500	Internal Server Error	Внутренняя ошибка сервера

5.3 Функционально тестирование клиентского приложения

Методика тестирования осуществлялась по принципу *черного ящика*, при котором проверялась работа системы без анализа внутренней реализации кода. Тестирование выполнялось параллельно с процессом разработки и включало ручную проверку клиентской частей приложения. Для комплексной оценки работоспособности были разработаны тестовые сценарии, охватывающие основные функции автоматизированной системы. В ходе тестирования были реализованы и проверены механизмы валидации пользовательских действий.

На рисунке 5.7 изображён механизм проверки перед добавлением позиции меню в заказ. Приложение сначала проверяет, выбран ли текущий заказ: если идентификатор заказа отсутствует, действие блокируется, а пользователь получает уведомление. Аналогичные проверки реализованы на сервере в контроллерах, что предотвращает отправку неполных или некорректных данных.

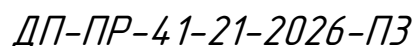
Инв. № подл.	Подп. и дата				
	Взам. инв. №				
	Инв. № дубл.				
	Подп. и дата				
Инв. № подл.	Лист	ДП-ПР-41-21-2026-ПЗ			
	37				
	Ли	Изм.	№ докум.	Подп.	Дата

Инв. № подл.	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата



Рисунок 5.8 - Проверка стола.

На рисунке 5.9 показан механизм удаления блюд из архива, а именно, что кнопки для удаления спрятаны. Данная проверка важна для корректного расчёта



конечной выручки, так как обеспечивает точное суммирование всех заказов без учёта удалённых позиций.

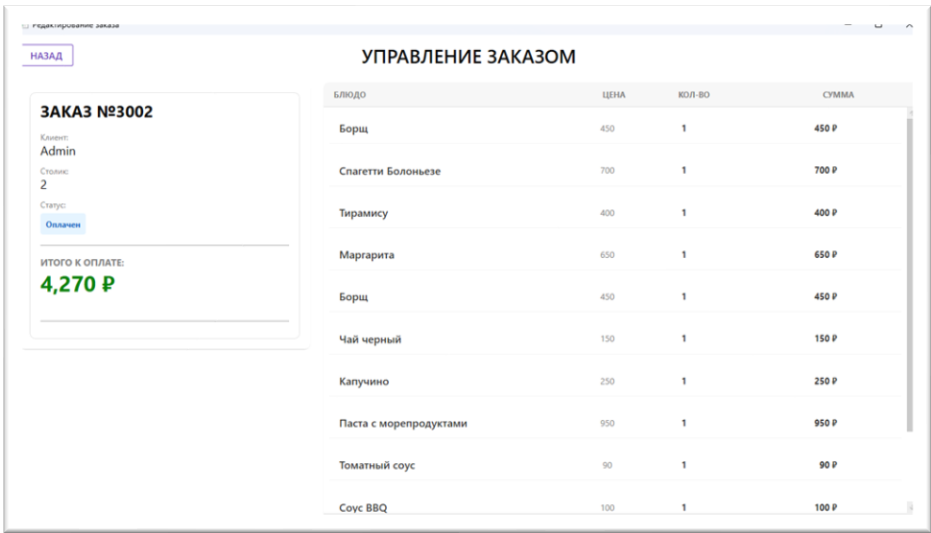


Рисунок 5.9 - Удаление из архива

На рисунке 5.10 показан механизм проверки статуса заказа. Если заказ успешно выполнен или оплачен, он перемещается в архив также перенося с собой статус и перестаёт отображаться на странице кухни, что обеспечивает реальную актуальность информации-вывода блюд для персонала.

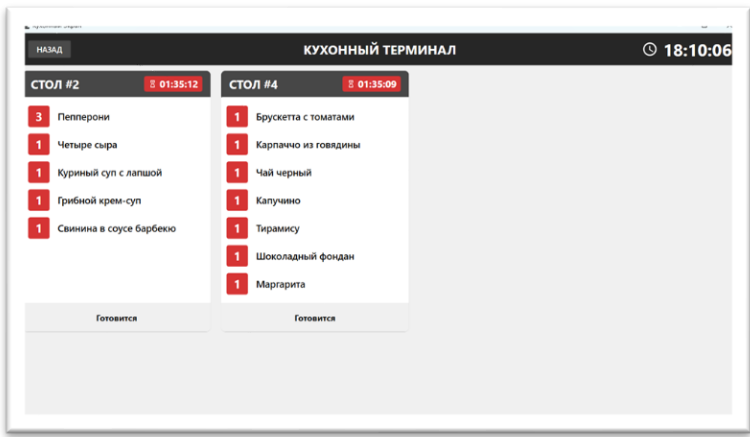


Рисунок 5.10 - Статус заказа кухня

Таким образом, использование стандартных HTTP-кодов позволяет упростить обработку результатов запросов как на стороне сервера, так и на стороне клиентского приложения. Это упрощает реализацию логики обработки ошибок, повышает читаемость кода и делает систему более устойчивой к некорректным входным данным и ошибкам выполнения.

6 Применение

6.1 Назначение программы

Приложение предназначено для использования на персональных компьютерах с операционной системой Windows и обеспечивает автоматизацию процессов обслуживания клиентов кафе.

Система поддерживает три роли пользователей. Администратор имеет полный доступ к функционалу системы: управление каталогом меню, регистрация и управление учетными записями сотрудников, формирования и экспорт отчетов о выручке за день и месяц. Официант имеет доступ к функциям, необходимым для работы с заказами. Повар имеет доступ к экрану кухни: просмотр активных заказов в реальном времени.

6.2 Требования к аппаратным ресурсам ПК.

Требования к компьютеру клиента (официант/повар/администратор):

- операционная система: Windows 10 / Windows 11
- оперативная память: 4,00 ГБ
- свободное место в хранилище: 100 МБ
- наличие сетевого подключения к серверу системы
- установленная среда выполнения: .NET 9

Требования к серверу (для развёртывания серверной части и базы данных):

- Операционная система: Windows 10, Windows 11, Linux.
- Оперативная память: 8,00 ГБ
- Свободное место в хранилище: 5 ГБ (Docker-образы весят ~2–3 ГБ, плюс данные БД)
- Установленный Docker Desktop / Docker Engine
- Наличие сетевого подключения

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата						Лист 40
Ли	Изм.	№ докум.	Подп.	Дата	ДП-ПР-41-21-2026-ПЗ					

6.3 Руководство пользователя.

При запуске приложения пользователя встречает экран авторизации, на рисунке 6.1. Для входа в систему необходимо ввести логин и пароль, после чего нажать кнопку «Войти». Система проверяет учётные данные и, в случае успеха, предоставляет доступ с ролью сотрудника.

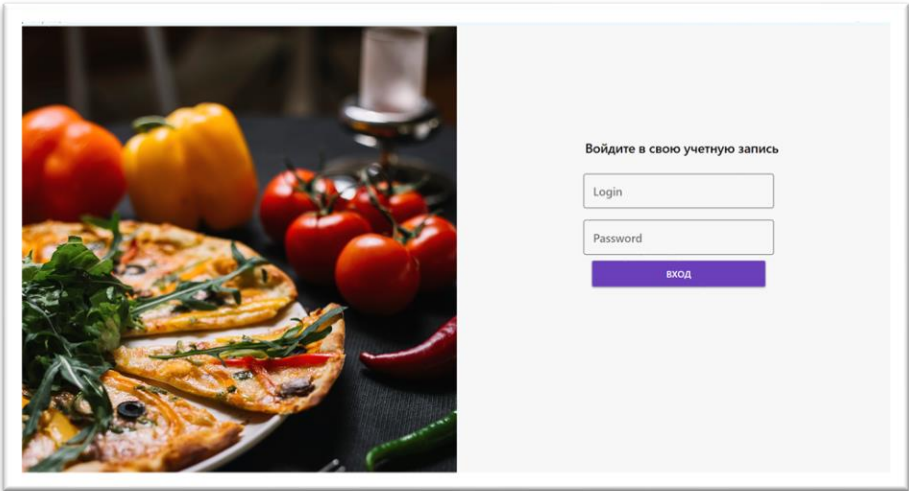


Рисунок 6.1 - Вход в приложение

После успешной авторизации открывается главная страница, рисунок 6.2. Интерфейс разделён на три раздела: список активных и архивных заказов, поле поиска блюд, каталог меню с фильтрацией по категориям. В верхней части находится меню с функциями, доступность которых зависит от роли пользователя.

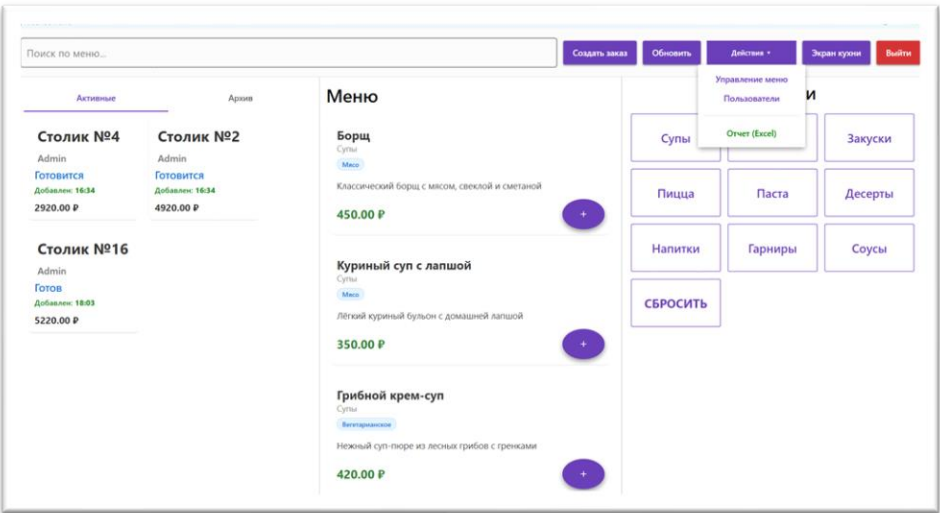


Рисунок 6.2 - Главный экран

Подп. и дата	
Взам. инв. №	
Инв. № дубл.	
Подп. и дата	
Инв. № подл.	

Ли	Изм.	№ докум.	Подп.	Дата

Для создания нового заказа необходимо в меню нажать «Создать заказ». Откроется страница с картой зала, на которой отображены столики. Занятые столики выделены красным цветом и недоступны для выбора. Для оформления заказа необходимо нажать на свободный столик. Представлено на рисунке 6.3



Рисунок 6.3 - Создание заказа

После выбора столика открывается главное меню, где двойным нажатием выбирается стол. Официант добавляет блюда из каталога, при необходимости изменяет их количество или удаляет позиции. На данной странице доступна функция разделения счёта. По завершению обслуживания необходимо нажать «Заккрыть счёт», после чего заказ переходит в статус «Оплачен» и перемещается в архив. Представлено на рисунке 6.4.

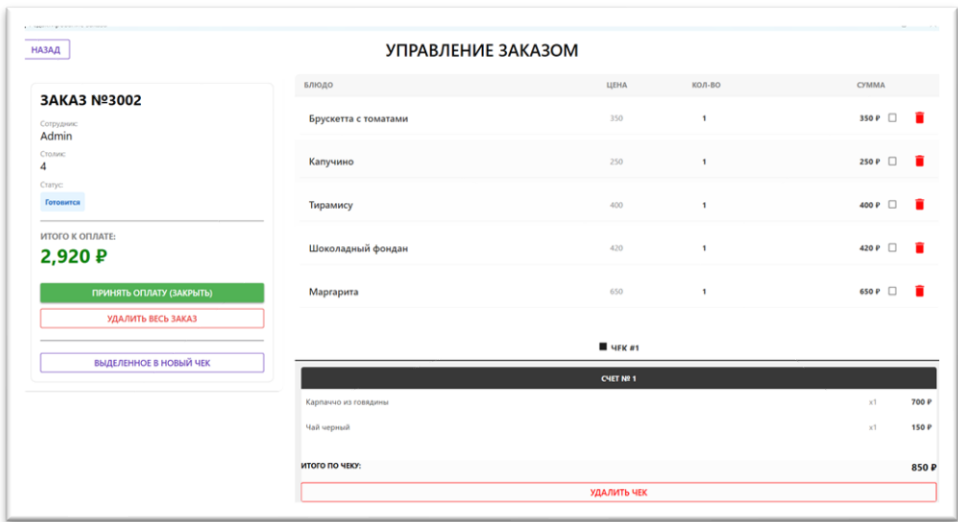


Рисунок 6.4 - Редактирование

Подп. и дата	
Взам. инв. №	
Инв. № дубл.	
Подп. и дата	
Инв. № подл.	

Ли	Изм.	№ докум.	Подп.	Дата
----	------	----------	-------	------

Для просмотра подробной информации о блюде необходимо нажать на него в каталоге. Откроется карточка блюда с изображением, названием, описанием, категорией, ценой и составом. Представлено на рисунке 6.5.

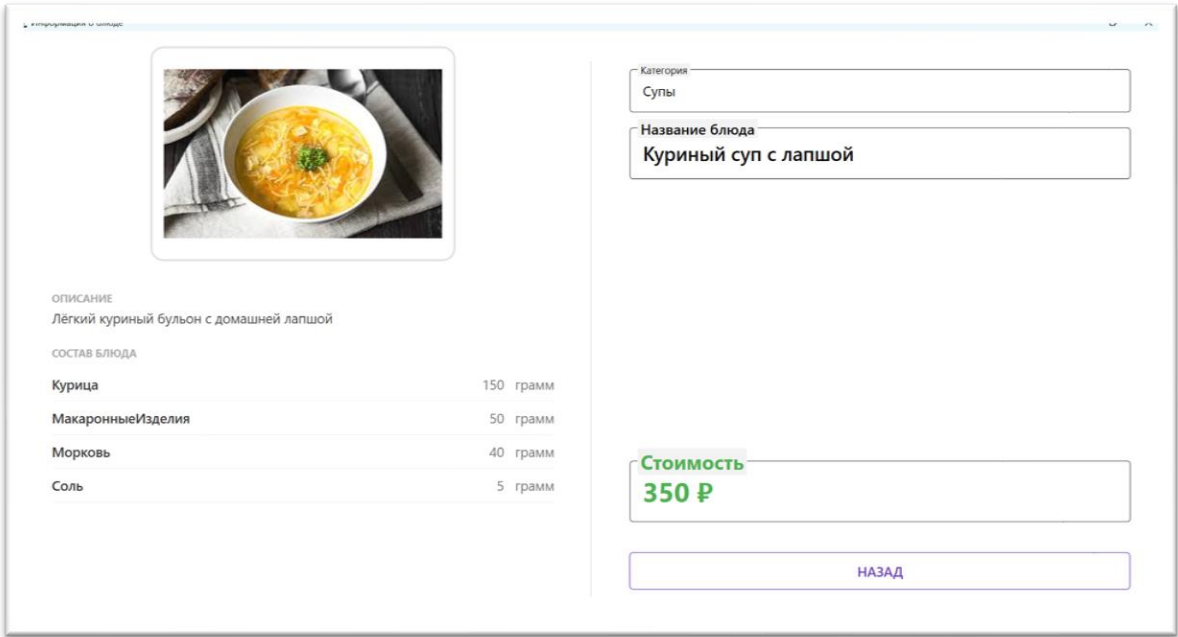
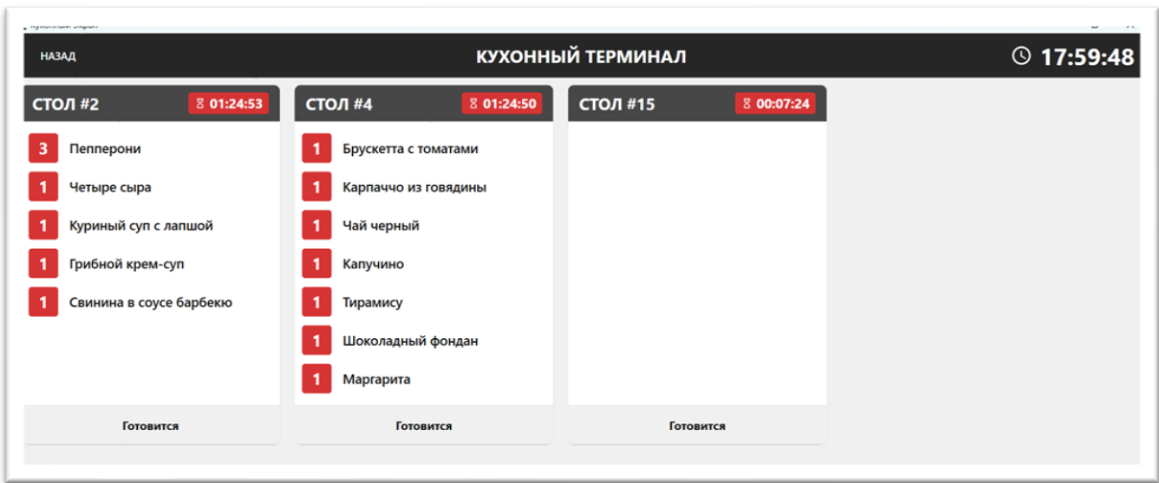


Рисунок 6.5- Карта блюда

Повар работает на экране кухни, а именно рисунке 6.6. На нём в реальном времени отображается список активных заказов. Каждый заказ представлен отдельной карточкой с перечнем блюд и их количеством. После приготовления заказа повар нажимает на статус заказа, переводя его в состояние «Готов», что автоматически уведомляет официанта о необходимости выдачи блюд.



Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата



Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата



Инв. № подл	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Для управления персоналом администратор переходит на страницу «Управление пользователями». Здесь отображается список всех сотрудников системы. Администратор может зарегистрировать нового сотрудника, указав его логин, пароль и роль, а также изменить данные существующего сотрудника или удалить его учётную запись, демонстрация экрана на рисунке 6.9.

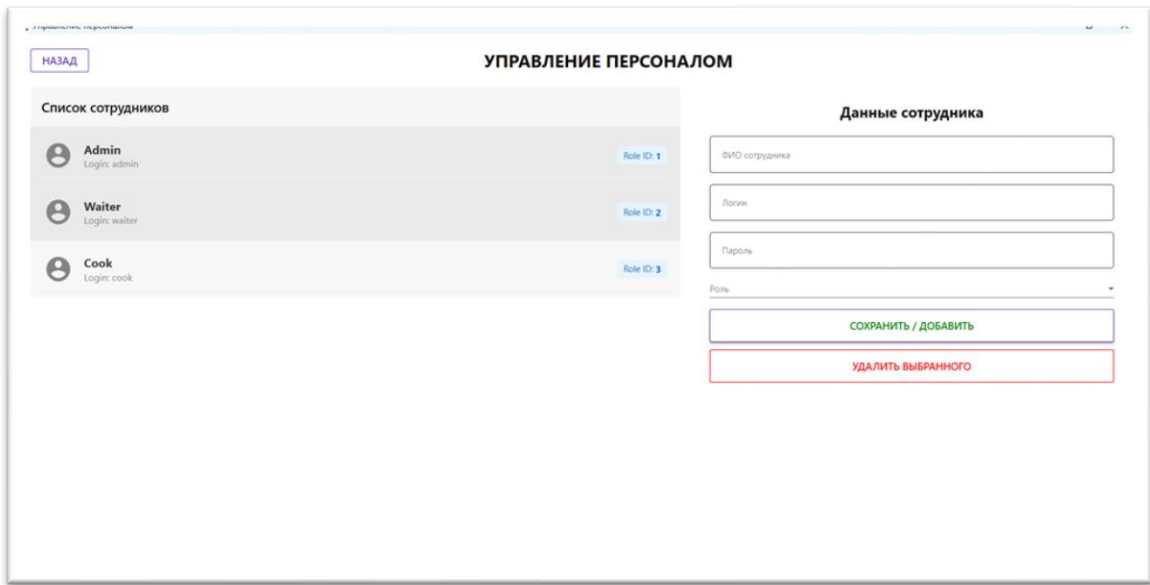


Рисунок 6.9 - Управление персоналом

Для выхода из системы необходимо в верхнем меню нажать «Выйти». Приложение завершает сессию (Данные о роле) и возвращает пользователя на экран авторизации. На рисунке 6.10 представлена кнопка «Выйти».

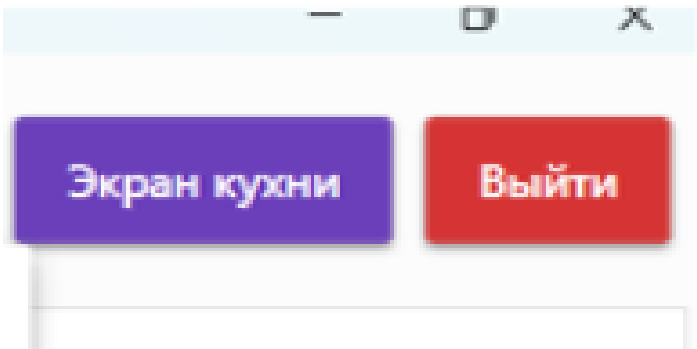


Рисунок 6.10 - Выход

В таком случае данные пользователя сбрасываются, а приложение забывает токен пользователя с данными, чтобы в случае чего не подменить его.

7 Охрана труда и противопожарная безопасность

Вопросы, относящиеся к обеспечению охраны труда при работе за компьютером, регулируются Федеральным законом от 17 июля 1999 г. №181-ФЗ «Об основах охраны труда в Российской Федерации» (далее - Закон об охране труда) и Санитарными правилами и нормами СанПиН 2.2.2.542-96 «Гигиенические требования к видео-дисплейным терминалам, персональным электронно-вычислительным машинам и организации работы».

Таким образом, ответственность за несоблюдение требований законодательства к условиям труда несёт работодатель, возлагающий эти функции на службу охраны труда организации или на привлечённого на договорных началах специалиста по охране труда.

Все негативные факторы при работе за компьютером можно поделить на такие основные группы:

- факторы, влияющие на опорно-двигательный аппарат;
- факторы, влияющие на сенсорные органы;
- факторы, влияющие на психологическое состояние.

Все эти факторы вызваны двумя основными причинами: неправильной работой за оборудованием и неправильным выбором оборудования

Несоблюдение требований безопасности приводит к тому, что спустя некоторое время работы за компьютером сотрудник начинает ощущать определённый дискомфорт: у него возникает головные боли и резь в глазах, появляются усталость и раздражительность. У некоторых людей нарушается сон, ухудшается зрение, начинают болеть руки, шея, поясница и т.д.

К наиболее распространённым ошибкам, связанным с обеспечением условий труда, работающих на компьютерах, относятся:

- недостаточные площадь и объём производственного помещения;
- несоблюдение требований, предъявляемых к температуре и влажности рабочих помещений;

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.						Лист 46
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Ли	Изм.	№ докум.	Подп.	Дата	ДП-ПР-41-21-2026-ПЗ

коэффициент естественной освещённости (КЕО) не ниже 1,2% в зонах с устойчивым снежным покровом и не ниже 1,5% на остальной территории. Указанные значения КЕО нормируются для зданий, расположенных в третьем световом климатическом поясе.

Не допускается располагать рабочие места для работы на компьютерах в подвальных помещениях. В случае производственной необходимости использовать помещения без естественного освещения можно только по согласованию с органами и учреждениями Государственного санитарно-эпидемиологического надзора России.

Площадь на одно рабочее место для взрослых пользователей должна быть не менее 6 м², а объём - не менее 23 м³.

Для внутренней отделки помещений должны использоваться диффузно - отражающие материалы с коэффициентом отражения от потолка - 0,7 - 0,8; для стен 0,5-0,6; для пола - 0,3-0,5. Полимерные материалы для внутренней отделки должны быть разрешены для применения органами и учреждениями Госсанэпиднадзора России.

Поверхность пола в помещениях должна быть ровной, без выбоин, нескользкой, удобной для очистки и влажной уборки, обладать антистатическими свойствами.

В производственных помещениях, в которых установлены компьютеры, микроклимат должен соответствовать следующим санитарным нормам:

- температура воздуха в тёплый период года - не более 23-25°C, в холодный - 22-24°C;
- относительная влажность воздуха - 40-60%;
- скорость движения - 0,1 м/с.

Для повышения влажности воздуха в помещениях следует применять увлажнители воздуха, ежедневно заправлять их дистиллированной или кипячённой водой.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	Инв. № подл.						Лист 48
Ли	Изм.	№ докум.	Подп.	Дата	ДП-ПР-41-21-2026-ПЗ						

Уровень положительных и отрицательных аэрофонов в воздухе помещений должен соответствовать «Санитарно-гигиеническим нормам допустимых уровней ионизации воздуха производственных и общественных помещений».

В производственных помещениях уровень шума на рабочих местах не должен превышать значений, установленных «Санитарными нормами допустимых уровней шума на рабочих местах», а уровень вибрации - «Санитарными нормами вибрации рабочих мест».

В помещениях, где эксплуатируются компьютеры, искусственное освещение должно быть общим и равномерным. Однако если сотрудники преимущественно работают с документами, то допускается применение комбинированного освещения: кроме общего устанавливаются светильники местного освещения, которые не должны создавать бликов на поверхности экрана и увеличивать его освещённость более 300 лк.

Освещённость поверхности стола в зоне размещения рабочего документа должна составлять 300-500 лк.

Источники освещения следует устанавливать таким образом, чтобы они не ослепляли, при этом яркость светящихся поверхностей (окна, светильники и др.), находящихся в поле зрения, должна быть более 200 кд/мІ.

В качестве источников света при искусственном освещении должны применяться преимущественно люминесцентные лампы типа ЛБ. При устройстве отражённого освещения допускается применение металло-галогенных ламп мощностью до 250 Вт, а в светильниках местного освещения - ламп накаливания.

Для обеспечения нормируемых значений освещённости в помещениях следует не реже двух раз в год чистить стёкла, оконные рамы и светильники и своевременно заменять перегоревшие лампы.

Рабочие места должны располагаться таким образом, чтобы естественный свет падал сбоку, преимущественно слева.

Расстояние между рабочими столами с мониторами (в направлении тыла поверхности одного монитора и экрана другого) должно быть не менее 2м, а между боковыми поверхностями мониторов - не менее 1,2м.

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	ДП-ПР-41-21-2026-ПЗ	Лист
Ли	Изм.	№ докум.	Подп.	Дата		49

Оконные проёмы должны быть оборудованы регулируемыми жалюзи, занавесями, внешними козырьками и др.

Желательно, чтобы высоту рабочей поверхности стола можно было регулировать в пределах 680-800мм, а при отсутствии такой возможности она должна быть равна 725мм. Модульными размерами рабочей поверхности компьютерного стола, на основании которых рассчитывают конструктивные размеры, следует считать: ширину 800, 1000, 1200, и 1400мм; глубину 800 и 1000мм.

Рабочий стол должен иметь пространство для ног высотой 600мм, шириной - не менее 500мм, глубиной на уровне колен - не менее 450мм, а на уровне вытянутых ног - не менее 650мм.

Конструкция рабочего стола должна обеспечивать оптимальное размещение на рабочей поверхности используемого оборудования с учётом его количества и конструктивных особенностей. Допускается использовать столы различных конструкций, соответствующих современным требованиям эргономики.

Конструкция рабочего стула (кресла) должна обеспечивать поддержание рациональной рабочей позы, позволять изменять её с целью снижения статистического напряжения мышц шейно-плечевой области и спины для предупреждения утомления.

Рабочий стул (кресло) должен быть подъёмно-поворотным, его высота и углы наклона сиденья и спины, а также расстояние спинки от переднего края сиденья должны независимо и легко регулироваться и иметь надёжную фиксацию, размеры рабочего стула приведены в СанПиН 2.2.2.542-96.

Поверхность сиденья, спинки и других элементов стула (кресла) должна быть полумягкой с нескользящим, не электризующимся и воздухопроницаемым покрытием, обеспечивающим лёгкую очистку от загрязнений.

Экран монитора должен находиться от глаз пользователя на оптимальном расстоянии 600-700мм, но не ближе 500мм с учётом размеров алфавитно-цифровых знаков и символов.

Инв. № подл.	Подп. и дата	Инв. № дубл.	Взам. инв. №	Подп. и дата						Лист 50
Ли	Изм.	№ докум.	Подп.	Дата	ДП-ПР-41-21-2026-ПЗ					

На рабочем месте устанавливается легко перемещаемый пюпитр для документов.

В помещении с компьютерами должна проводиться ежедневная влажная уборка. Они должны быть оснащены аптечкой первой помощи и углекислотными огнетушителями.

Режимы труда и отдыха при работе на компьютерах зависят от вида и категории трудовой деятельности.

СанПиН 2.2.2.542-96 устанавливает категории тяжести и напряжённости работы на компьютерах, которые определяются: для группы А - по суммарному числу считываемых знаков за рабочую смену, но не более 60 тыс. знаков за смену; для группы Б - по суммарному числу считываемых или вводимых знаков за рабочую смену, но не более 40 тыс. знаков за смену.

Время регламентированных перерывов в течение рабочей смены устанавливается в зависимости от продолжительности рабочей смены, вида и категории трудовой деятельности:

Виды трудовой деятельности разделяются на три группы:

- группа А - работа по считыванию информации с экрана монитора с предварительным запросом;
- группа Б - работа по вводу информации;
- группа В - творческая работа в режиме диалога с ЭВМ.

При выполнении в течение рабочей смены работ, относящихся к разным видам трудовой деятельности, за основную следует принимать такую, которая занимает не менее 50% времени в течение рабочей смены или рабочего дня.

Вид трудовой деятельности, тяжесть и напряжённость работ устанавливается на основе аттестации рабочих мест по условиям труда. Как правило, работа сотрудников отделов кадров относится к группам А и Б.

При восьмичасовой рабочей смене в работе на компьютере регламентированные перерывы следует устанавливать:

- для I категории работ - через 2 часа от начала рабочей смены и через 2 часа после обеденного перерыва продолжительностью по 15 мин.;

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата	Инв. № подл.						ДП-ПР-41-21-2026-ПЗ	Лист 51
Ли	Изм.	№ докум.	Подп.	Дата								

- для II категории работ - через 2 часа от начала рабочей смены и через 1,5 - 2 часа после обеденного перерыва продолжительностью по 15 мин. или через каждый час работы продолжительностью по 10 мин.

Во время регламентируемых перерывов с целью снижения нервно-эмоционального напряжения, уменьшения утомления глаз, устранения гиподинамики и гипокинезии целесообразно выполнять комплексы упражнений, изложенных в СанПиН 2.2.2.542-96.

Инструкция по технике безопасности:

- при использовании компьютером следует носить чистую и сухую одежду и обувь;
- если монитор не имеет защиты от излучения, следует пользоваться защитным экраном. Излучение монитора в сторону противоположную экрану может быть значительно больше, поэтому нельзя его заднюю часть обращать к соседям по офису или комнате;
- при обнаружении неисправности ПЭВМ или появлении необычных звуков в процессе работы следует выключить компьютер;
- для устранения последствий скачков напряжения в сети, компьютер должен быть подключен к электросети через стабилизатор напряжения (бесперебойный источник питания).

Запрещается:

- включать и выключать компьютер без необходимости (это может привести к его неисправности);
- трогать разъемы соединительных кабелей, проводов, вилки и розетки;
- прикасаться к экрану и к тыльной стороне блоков компьютера;
- работать на ПЭВМ мокрыми руками;
- работать на ПЭВМ, имеющих нарушения целостности корпуса, нарушения изоляции проводов, неисправную индикацию включения питания, с признаками электрического напряжения на корпусе.
- класть на ПЭВМ посторонние предметы (кружки с жидкостями, жирные предметы, книги и предметы, излучающие электромагнитные поля).

Заключение

В ходе работы дипломного проекта - разработана автоматизированная система для кафе. Программа решает главные проблемы малого и среднего ресторанного бизнеса. Решает проблемы ручных действий, из-за которых сотрудники часто путают или забывают заказы. Также система решает проблему связи с кухней и делает денежную отчетность адекватно прозрачной в реальном времени. В итоге получилось приложение, которое связывает официантов, поваров и администратора в одно информационное пространство.

На начальном этапе был проведен анализ предметной области и было выполнено сравнение разрабатываемой системы с популярными аналогами, такими как iiko и R-Keper. Были выявлены недостатки - высокая стоимость лицензий, дорогая подписка и завышенные требования к оборудованию. Из-за архитектуры плохо работают на простых офисных ПК. Разработанный продукт решает эту проблему. На основе анализа было спроектировано клиент-серверное приложение. Проект построен по паттерну N-Layer (многослойная архитектура). Это позволило чётко разделить зоны ответственности между компонентами и обеспечило независимость базы данных, бизнес-логики и интерфейса. В будущем такая система не будет требовать переписывание всей архитектуры бэкенд или десктопного приложения.

Для логической структуры спроектирована и реализована база данных на базе СУБД Microsoft SQL Server. Архитектура включает в себя 9 таблиц для хранения данных о сотрудниках, их ролях, категориях меню, пунктах каталога, ингредиентах и составе блюд. Также в базе сохраняется история активных и архивных заказов. Взаимодействие сервера с базой данных организовано через Entity Framework Core. Это упростило работу с генерацией SQL-запросов и обеспечило обыкновенную типизацию на уровне кода. Использование реляционной модели позволило обеспечить согласованность информации при одновременной работе нескольких пользователей.

Функциональное тестирование WPF-приложения велось по принципу чёрного ящика. Проверка подтвердила, что интерфейс устойчив к ошибкам

пользователей, валидация данных работает правильно, а стандартные HTTP-коды ответов сервера обрабатываются корректно.

В дальнейшем разработанная автоматизированная система может быть расширена и доработана. Одним из направлений развития куда можно вложить силы это адаптация ПО для работы на POS-терминалах, то есть полное тестирование программы в ресторане на множестве ПК и оборудования чтобы выяснить какие неполадки есть, а что сделать для заказчика. Также возможно внедрение системы онлайн-бронирования столиков через приложение или же сайт.

Реализация данного функционала покажет, что можно пользоваться не только услугами популярных сервисов, но и системами абсолютно сделанных под требования заказчика.

Инв. № подл	Подп. и дата	Инв. № дудл	Взам. инв. №	Подп. и дата						
Ли	Изм.	№ докум.	Подп.	Дата	ДП-ПР-41-21-2026-ПЗ					Лист
										55

Список использованных источников

1. Макдональд М. WPF: Windows Presentation Foundation в .NET 4.5 с примерами на C# 5.0 для профессионалов / М. Макдональд. – Москва: Вильямс, 2013. – 1024 с.
2. Мартин Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – Санкт-Петербург: Питер, 2021. – 352 с.
3. Прайс М. C# 9 и .NET 5. Разработка и оптимизация / М. Прайс. - Санкт-Петербург: Питер, 2022. – 848 с.
4. Троелсен Э. Язык программирования C# 9 и платформа .NET 5 / Э. Троелсен, Ф. Джепикс. – Москва: Вильямс, 2022. – 1328 с.
5. Фримен А. ASP.NET Core для профессионалов / А. Фримен. – Москва: Вильямс, 2019. – 912 с.
6. Документация по службе «Управление API» [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/azure/api-management/> (дата обращения 10.10.2025).
7. Документация по ASP.NET Core [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/aspnet/core/> (дата обращения 10.11.2025).
8. Документация по Entity Framework Core [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/ef/core/> (дата обращения 11.11.2025).
9. Руководство по языку C# [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/csharp/> (дата обращения 13.01.2026).

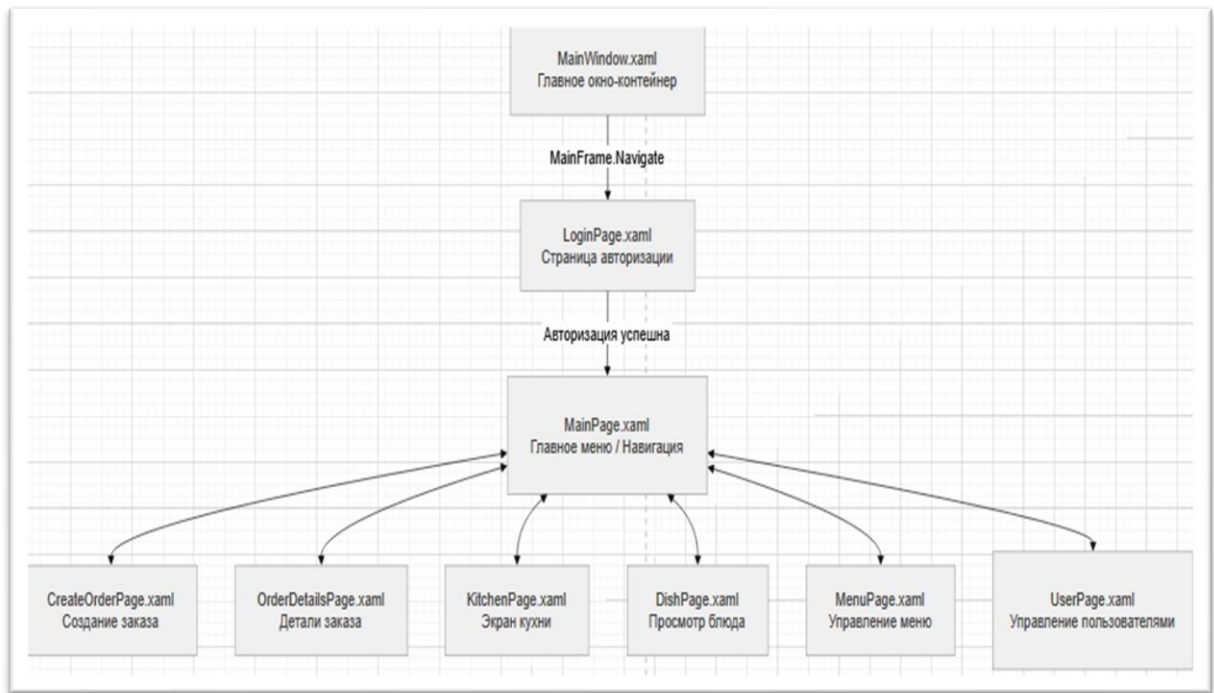
[illegible]

Приложения

Инв. № подл	Подп. и дата				Взам. инв. №	Инв. № дудл.	Подп. и дата	Инв. № подл	ДП-ПР-41-21-2026-ПЗ					Лист
														57
	Ли	Изм.	№ докум.	Подп.										Дата

Приложение А Схема алгоритма

Инв. № подл	Подп. и дата		Взам. инв. №	Инв. № дудл.	Подп. и дата	Инв. № подл	ДП-ПР-41-21-2026-ПЗ					Лист
												58
	Ли	Изм.										№ докум.



Карта навигации приложения

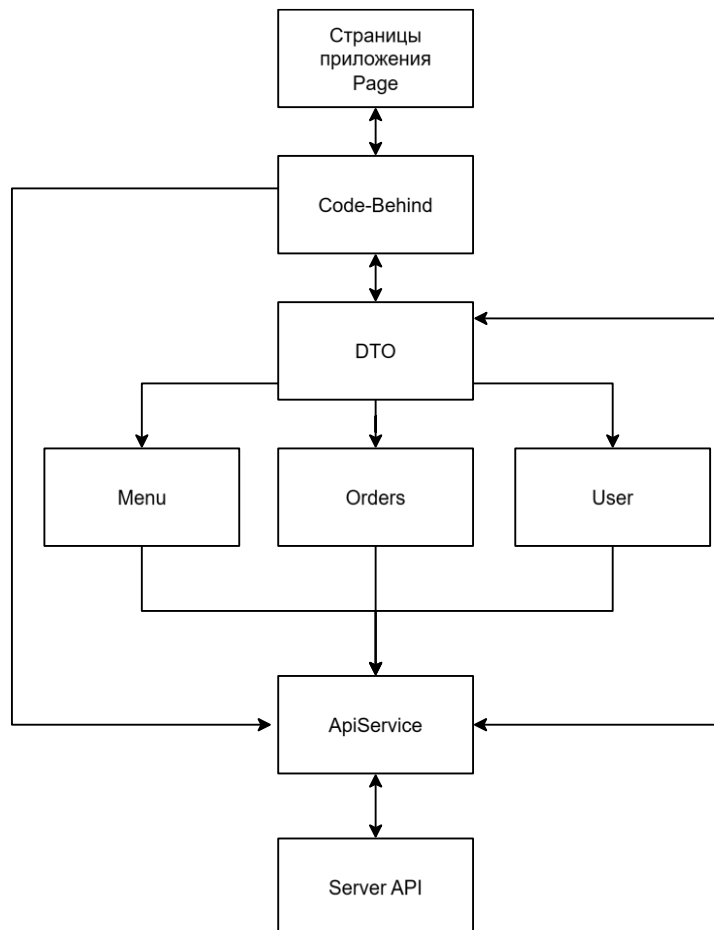
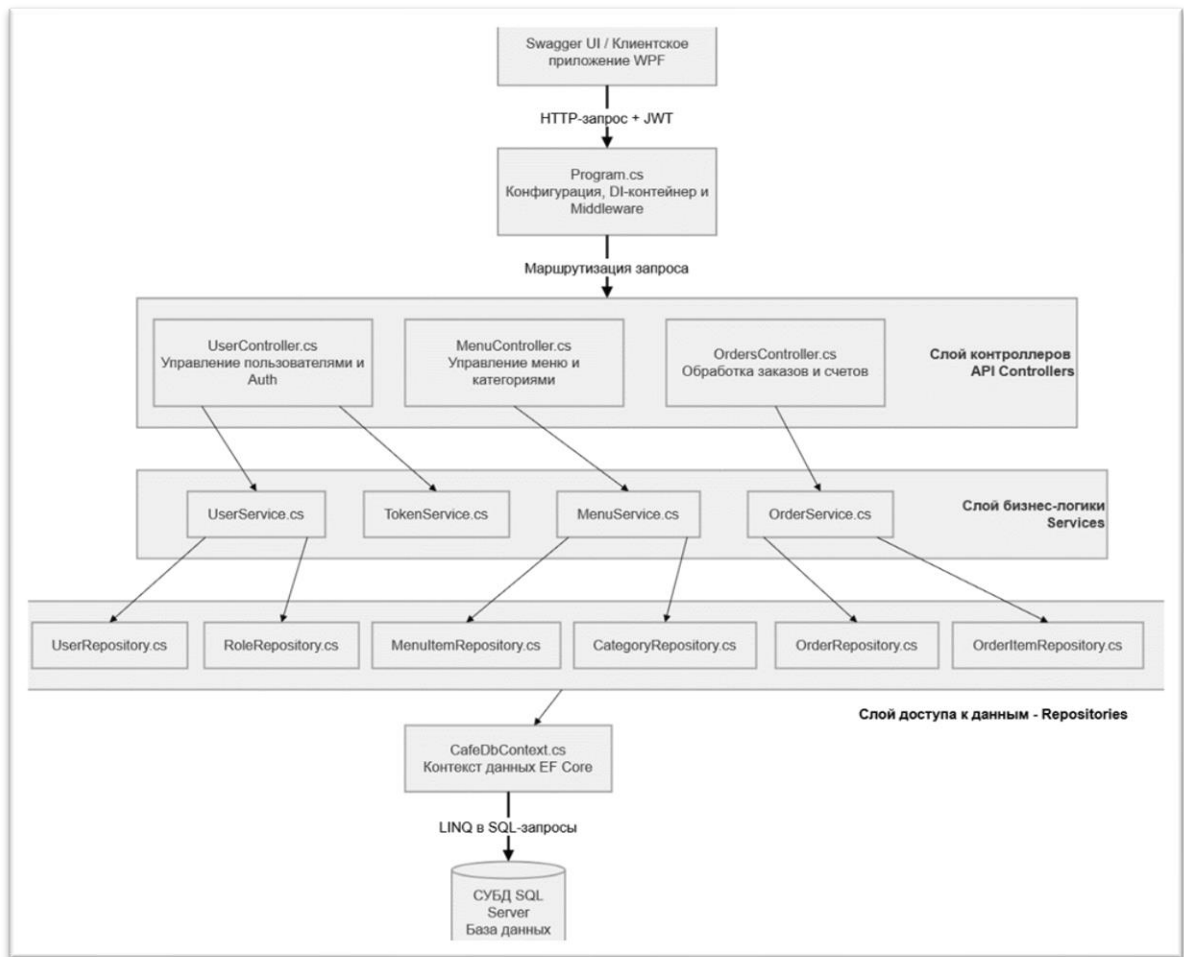


Схема работы архитектуры приложения

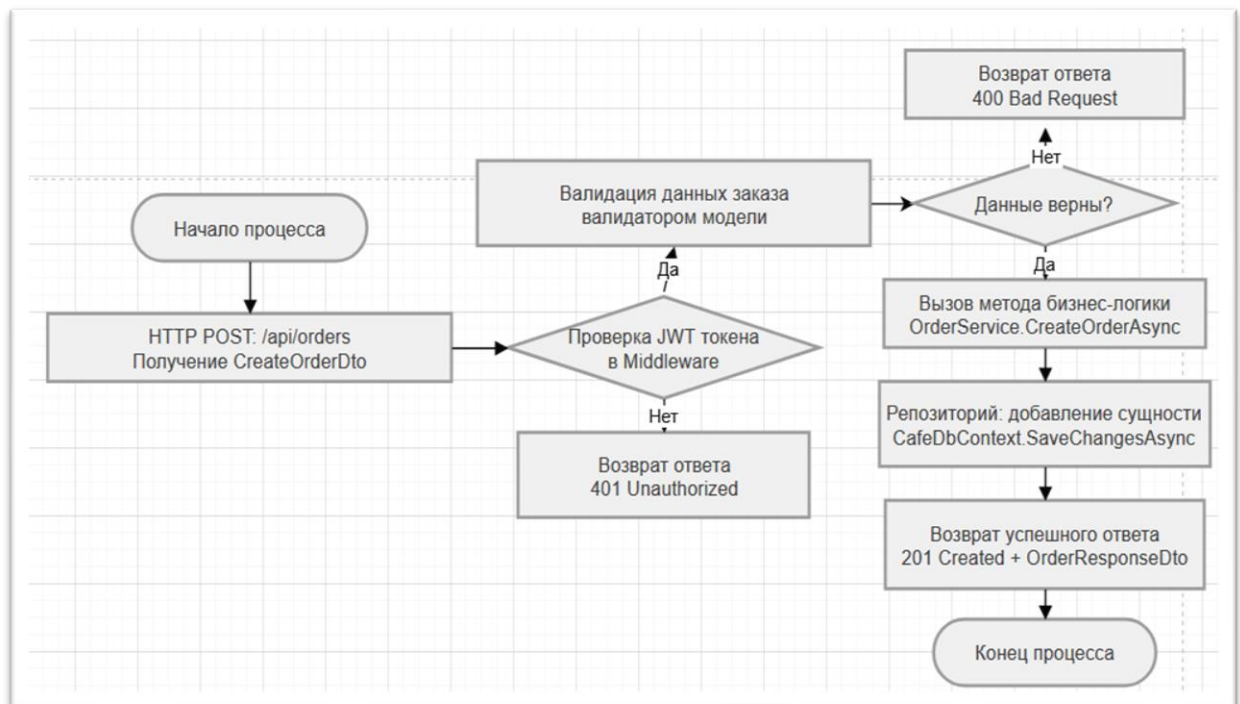
Инв. № подл	Подп. и дата
Инв. № докл.	Взам. инв. №
Подп. и дата	Инв. № инв.
Инв. № подл	Подп. и дата

Ли	Изм.	№ докум.	Подп.	Дата

ДП-ПР-41-21-2026-ПЗ



Бэкенд схема серверной части



Блок схема создания заказа

Инв. № подл.	Подп. и дата
Инв. № докл.	Взам. инв. №
Подп. и дата	Подп. и дата
Инв. № подл.	Взам. инв. №

Ли	Изм.	№ докум.	Подп.	Дата

Приложение Б Текст программы

Инв. № подл	Подп. и дата		Инв. № дудл	Взам. инв. №	Подп. и дата	
Ли	Изм.	№ докум.	Подп.	Дата	ДП-ПР-41-21-2026-ПЗ	Лист
						61

Репозиторий проекта: <https://github.com/Bikmacs/CafeSystem>



QRCode репозитория проекта



QRCode документация API через Swagger UI

Инв. № подл	Подп. и дата	Инв. № докл.	Взам. инв. №	Подп. и дата

Ли	Изм.	№ докум.	Подп.	Дата

ДП-ПР-41-21-2026-ПЗ

Лист

62

Код из класса контроллера проекта

```
[Route("api/[controller]")]
[ApiController]
public class MenuController(IMenuService menuService) : ControllerBase
{
    [Authorize(Roles = "Admin,Waiter")]
    [HttpGet("GetMenu")]
    public async Task<IActionResult> GetMenu()
    {
        var result = await menuService.GetMenuAsync();
        return result.Any() ? Ok(result) : NotFound("Список пуст");
    }

    [Authorize(Roles = "Admin")]
    [HttpPost("Add")]
    public async Task<IActionResult> AddItem([FromBody] CreateMenuItemDto dto)
    {
        try
        {
            var result = await menuService.AddItemMenuAsync(dto);
            return CreatedAtAction(nameof(GetMenuItemById), new { id =
result.MenuItemId }, result);
        }
        catch (Exception ex)
        {
            return BadRequest(ex.Message);
        }
    }

    [Authorize(Roles = "Admin,Waiter")]
    [HttpDelete("{id}")]
    public async Task<IActionResult> DeleteItem(int id)
    {
        var success = await menuService.DeleteItemMenu(id);
        return success ? Ok("удалено") : NotFound("Не найдено");
    }

    [Authorize(Roles = "Admin,Waiter")]
    [HttpGet("{id}")]
    public async Task<IActionResult> GetMenuItemById(int id)
    {
        var item = await menuService.GetMenuItemById(id);
        return Ok(item);
    }

    [Authorize(Roles = "Admin")]
    [HttpPost("CreateProduct")]
    public async Task<IActionResult> AddProducts([FromQuery] CreateProductDto
productDto)
    {
        try
        {
            var unit = ProductUnitHelper.GetProductUnits[productDto.Product];

            return Ok(new
            {
                Message = $"Запасы пополнены: {productDto.Product} -
{productDto.Quantity} {unit}."
            });
        }
        catch (Exception e)
        {
        }
    }
}
```

Подп. и дата

Взам. инв. №

Инв. № дудл.

Подп. и дата

Инв. № подл.

Ли Изм. № докум. Подп. Дата

ДП-ПР-41-21-2026-ПЗ

Лист

63


```

        Console.WriteLine(e);
        throw;
    }
}

[Authorize(Roles = "Admin")]
[HttpPost("{id}/image")]
public async Task<IActionResult> UploadImage(int id, IFormFile file) // ← id,
не idm
{
    if (file == null || file.Length == 0)
        return BadRequest("Файл не выбран");

    var uploadFolder = Path.Combine(Directory.GetCurrentDirectory(),
"wwwroot", "images");
    if (!Directory.Exists(uploadFolder))
        Directory.CreateDirectory(uploadFolder);

    var fileName = $"{id}{Path.GetExtension(file.FileName)}";
    var filePath = Path.Combine(uploadFolder, fileName);

    using (var fileStream = new FileStream(filePath, FileMode.Create))
        await file.CopyToAsync(fileStream);

    var updateDto = new UpdateMenuItemDto { Image = fileName };
    await menuService.UpdateItemMenu(id, updateDto);

    return Ok(new { ImageUrl = $"/images/{fileName}" });
}

[Authorize(Roles = "Admin,Waiter")]
[HttpGet("GetCategories")]
public async Task<IActionResult> GetCategories()
{
    var result = await menuService.GetCategories();
    return result.Any() ? Ok(result) : NotFound("Список пуст");
}
}

```

Инв. № подл	Подп. и дата	Инв. № дудл	Взам. инв. №	Подп. и дата	ДП-ПР-41-21-2026-ПЗ					Лист
										64
Ли	Изм.	№ докум.	Подп.	Дата						

Приложение В Структура данных

Инв. № подл	Подп. и дата				Взам. инв. №	Инв. № дудл.	Подп. и дата	Инв. № подл	
Ли	Изм.	№ докум.	Подп.	Дата	ДП-ПР-41-21-2026-ПЗ				Лист
									65



Диаграмма базы данных



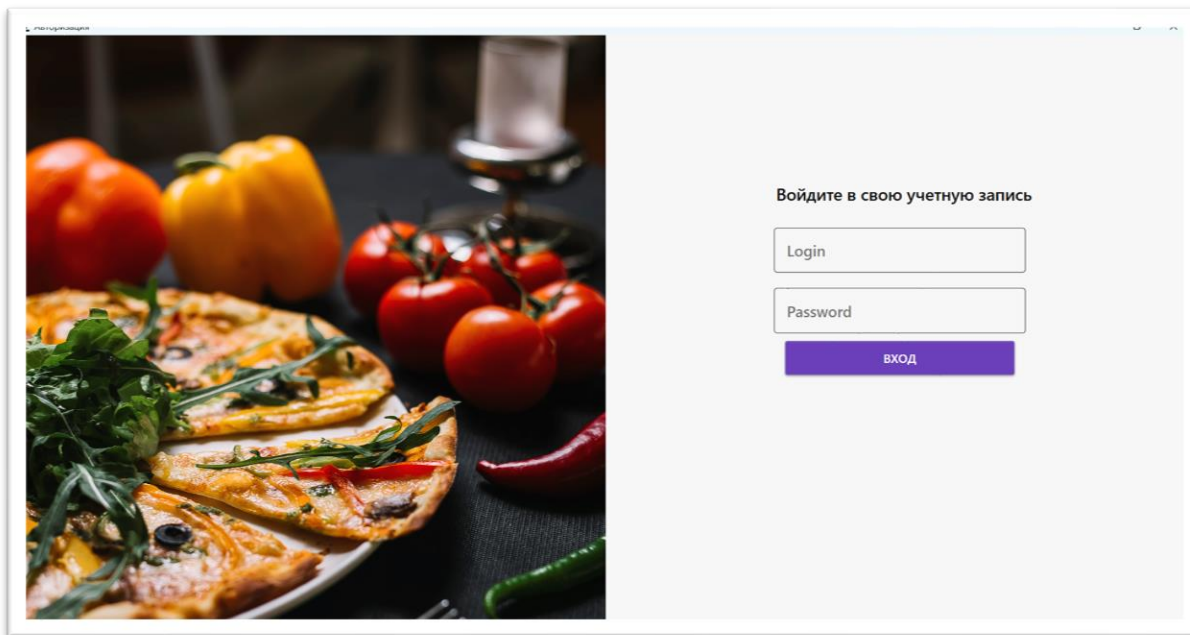
Qrcode - Данные для автоматического заполнения базы данных при инициализации

Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дудл.	Подп. и дата	Инв. № подл.
Ли	Изм.	№ докум.	Подп.	Дата	
ДП-ПР-41-21-2026-ПЗ					Лист
					66

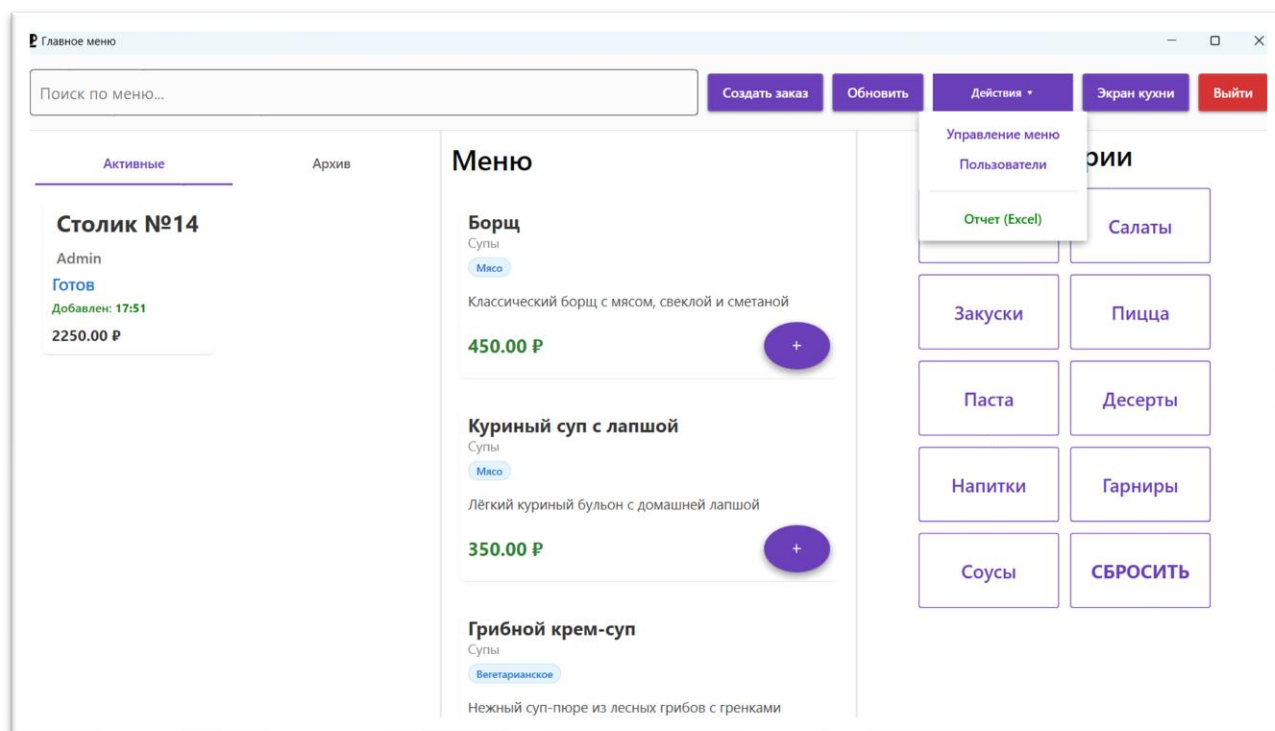
Приложение Г Скриншоты клиентского приложения

Инв. № подл.	Подп. и дата	Инв. № дудл.	Взам. инв. №	Подп. и дата

Ли	Изм.	№ докум.	Подп.	Дата	ДП-ПР-41-21-2026-ПЗ	Лист
						67



Экран страницы входа в приложение, где требуется пароль и логин

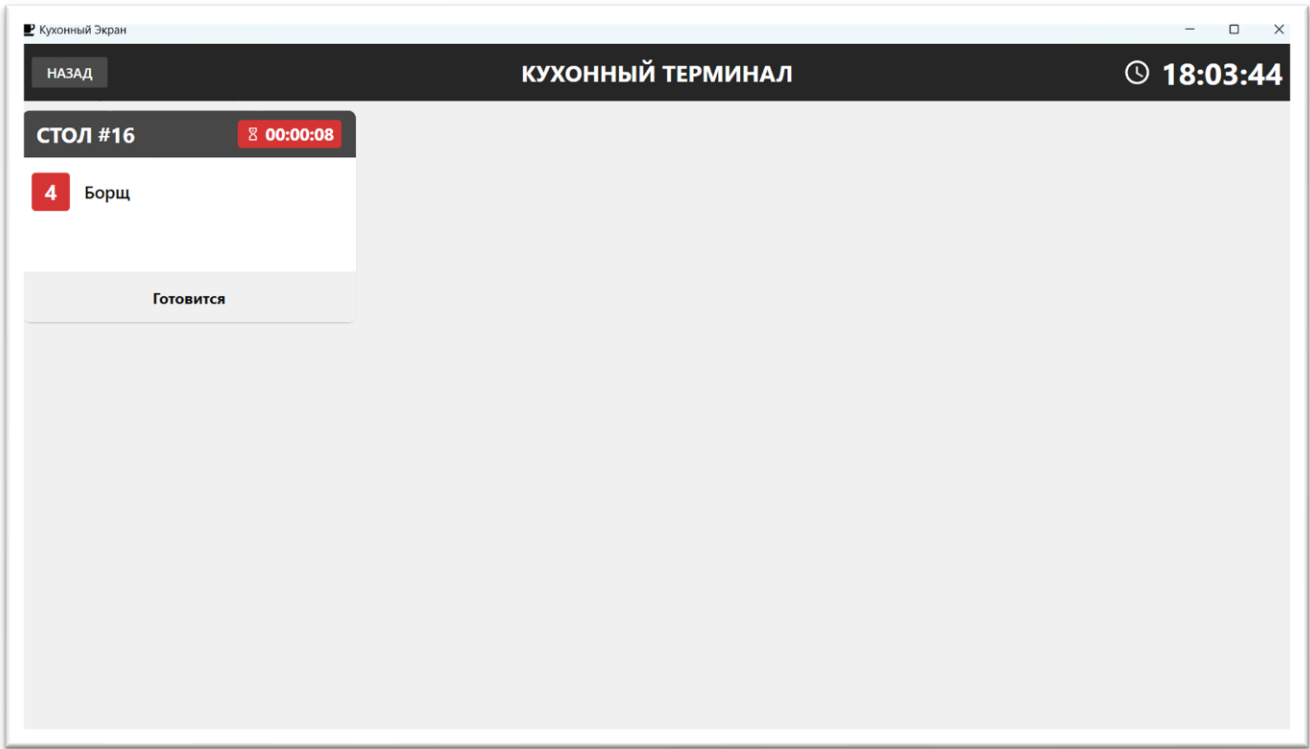


Главное меню приложения где доступен основной функционал программы

Инв. № подл.	Подп. и дата
Инв. № докл.	Взам. инв. №
Инв. № подл.	Подп. и дата
Инв. № подл.	Подп. и дата



Экран выбора столика, показано какие столы заняты и показана карта зала ресторана



Экран кухонного персонала, где появляются заказы в реальном времени

Подп. и дата	
Взам. инв. №	
Инв. № дудл.	
Подп. и дата	
Инв. № подл.	

Ли	Изм.	№ докум.	Подп.	Дата
----	------	----------	-------	------

Управление меню

НАЗАД

НОВОЕ БЛЮДО

Название блюда

Категория

Цена (₽)
0

Описание состава

☒ Доступно для заказа

ДОБАВИТЬ В МЕНЮ

ОЧИСТИТЬ ФОРМУ

УПРАВЛЕНИЕ МЕНЮ

Название	Описание	Цена	Категория	
авыа	авыа	1,234 Р	Закуски	
Сметанный соус	Нежный соус со сметаной и зеленью	80 Р	Соусы	
Томатный соус	Соус из томатов для пиццы и пасты	90 Р	Соусы	
Соус BBQ	Сладко-пряный соус для мяса	100 Р	Соусы	
Овощи на пару	Сезонные овощи на пару	230 Р	Гарниры	
Рис с овощами	Отварной рис с овощами	220 Р	Гарниры	
Картофель фри	Хрустящий картофель фри	250 Р	Гарниры	
Сок апельсиновый	Свежежатый апельсиновый сок	200 Р	Напитки	
Чай черный	Классический черный чай	150 Р	Напитки	

Экран управлением меню, где есть список всех блюд

Управление персоналом

НАЗАД

Список сотрудников

Admin

Login: admin

Role ID: 1

Waiter

Login: waiter

Role ID: 2

Cook

Login: cook

Role ID: 3

Данные сотрудника

ФИО сотрудника

Логин

Пароль

Роль

СОХРАНИТЬ / ДОБАВИТЬ

УДАЛИТЬ ВЫБРАННОГО

Экран управлением персоналом, где есть список всех сотрудников

Подп. и дата

Взам. инв. №

Инв. № дцкл.

Подп. и дата

Инв. № подл.